

Complier-4 보고서

I. 프로그램 수행 도중 발생한 오류 및 해결 방법

① gen_statement의 case N_STMT_FOR에서 중괄호를 잘못 배치했다.

- 해결 방법: 적절한 위치에 닫는 괄호의 위치를 수정하였다.

② gen_expression의 case N_EXP_EMP_MINUS에서 float 타입인 경우 MINUSF를 해야하는데 MINUS를 하였다.

- 해결 방법: 잘못 타이핑한 코드를 수정하였다.

(*이전 과제 수정 부분)

③ struct를 정의할 때, 자기 자신을 포함할 수 없다.

- 해결 방법: 선택스 분석 함수 중 getTypeIdOfStructOrEnumRefIdentifier에서 id->type->size가 0인 경우 선택스 에러가 발생하도록 하였다.

(*이전 과제 수정 부분)

④ N_FOR_EXP 노드가 llink를 제대로 인식하지 못하였다.

- 해결 방법: sem_for_expression에서 N_FOR_EXP인 경우 llink가 아닌 rlink의 타입을 검사하고 있는 부분을 발견하고, 해당 부분은 llink로 수정하였다.

(*이전 과제 수정 부분)

⑤ isPointerOrArrayType 함수의 중복 선언 오류가 발생하였다.

- 해결 방법: 시멘틱 분석 함수 isPointerOrArrayType 부분을 주석 처리 하였다.

*완성하지 못한 부분

실수 상수의 값 가져오는 부분이 값이 아닌 주소로 되어 있다.

1.

[입력값]

```
virtual-machine:~$ cat ex.c
int main(){
    int a;
    float b;
    char c;
    char *d;

    a=3;
    b=2.25;
    c='c';
    d="string";
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
===== semantic tree =====
N_PROGRAM (0,28)
| (ID="main") TYPE:3b002150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,16)
| | | | | | (ID="a") TYPE:3affcac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | TYPE
| | | | | | | (int)
| | | | | | (ID="b") TYPE:3affcb50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | TYPE
| | | | | | | (float)
| | | | | | (ID="c") TYPE:3affcbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
| | | | | | | TYPE
| | | | | | | (char 1)
| | | | | | (ID="d") TYPE:3b002380 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=24
| | | | | | | TYPE
| | | | | | | | POINTER
| | | | | | | | | ELEMENT_TYPE
| | | | | | | | | (char 1)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (3affcac0,0)
| | | | | | | | | N_EXP_IDENT (3affcac0,1)
| | | | | | | | | | (ID="a") TYPE:3affcac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_EXP_INT_CONST (3affcac0,0)
| | | | | | | | | | | INT=3
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (3affcb50,0)
| | | | | | | | | | N_EXP_IDENT (3affcb50,1)
| | | | | | | | | | | (ID="b") TYPE:3affcb50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | | N_EXP_FLOAT_CONST (3affcb50,0)
| | | | | | | | | | | LITERAL: 0.000000
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (3affcbe0,0)
| | | | | | | | | | N_EXP_IDENT (3affcbe0,1)
| | | | | | | | | | | (ID="c") TYPE:3affcbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
```

```
| | | | | (ID="c") TYPE:3affcbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20  
| | | | | N_EXP_CHAR_CONST (3affcbe0,0)  
| | | | | INT=99  
| | | | | N_STMT_LIST (0,0)  
| | | | | N_STMT_EXPRESSION (0,0)  
| | | | | N_EXP_ASSIGN (3b002380,0)  
| | | | | N_EXP_IDENT (3b002380,1)  
| | | | | (ID="d") TYPE:3b002380 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=24  
| | | | | N_EXP_STRING_LITERAL (3affccb0,0)  
| | | | | LITERAL: "string"  
| | | | | N_STMT_LIST_NIL (0,0)
```

start code generation

end code generation

```
INT 0, 28
SUP 0, main
RET 0, 0
```

INT	0, 28
LDA	1, 12
LITI	0, 3
STX	0, 0
POP	0, 1
LDA	1, 16
LOD	0, 12
STX	0, 0
POP	0, 1
LDA	1, 20
LITI	0, 99
STXB	0, 0
POP	0, 1
LDA	1, 24
LDA	0, 16
STX	0, 0
POP	0, 1
RET	0, 0

```
.literal    16  "string"
```

```
-virtual-machine:~$ ./interpreter a.asm

===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3

===== code =====
0: INT 0,28
1: SUP 0,3
2: RET 0,0
3: INT 0,28
4: LDA 1,12
5: LITI 0,3
6: STX 0,0
7: POP 0,1
8: LDA 1,16
9: LOD 0,12
10: STX 0,0
11: POP 0,1
12: LDA 1,20
13: LITI 0,99
14: STXB 0,0
15: POP 0,1
16: LDA 1,24
17: LDA 0,16
18: STX 0,0
19: POP 0,1
20: RET 0,0

start execution
end execution
```

[설명]

시멘틱 트리를 확인했을 때 작성한 코드가 제대로 트리를 출력하는 것을 확인할 수 있다. N_STMT_COMPOUND에 4byte 지역 변수 4개가 존재해 총 16이라는 value가 적혀있고, 각 지역변수의 주소와 레벨 값이 적절하게 들어있다. N_EXP_ASSIGN에서는 하위에 존재하는 identifier의 주소와 value가 1인 것을 확인할 수 있다. 여기서 1은 lvalue 여부로, lvalue이다. 코드로부터 발생한 리터럴 테이블 생성 내용은 총 2개(실수형 상수, 스트링 리터럴)가 존재한다. a.asm에서 확인할 수 있듯이, literal은 총 2개가 잘 입력되지만, 주소값의 변환 문제로 제대로 된 값이 들어 있지 않은 것을 확인할 수 있다. 스트링 리터럴은 적절한 결과를 보여준다. 어셈블리 코드를 확인 해보면, 함수에서 12+16(지역변수 4개)로 총 28만큼 INT되는 것을 확인할 수 있다. 이후에 총 4번의 ASSIGN 연산이 이루어지는데, 이때 변수의 레벨과 주소를 통해 해당 변수의 주소를 가져와서 LITI로 가져온 값을 STX, POP을 통해 저장하는 것을 확인할 수 있다. 최종 interpreter의 결과 역시도 symbol을 통해 적절하게 변형이 된 동일한 결과를 확인할 수 있다. 함수들은 실행 끝에 return(RET)하고 정상적으로 동작하는 것을 확인할 수 있다.

2.

[입력값]

```
int main(){
    int a[10];
    a[3]='c';
    printf("%c\n",a[3]);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
start semantic analysis
--- warning at line 3:incompatible types in assignment expression
===== semantic tree =====
N_PROGRAM (0,20)
| (ID="main") TYPE:1b838150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,40)
| | | | | | (ID="a") TYPE:1b838260 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | | ARRAY
| | | | | | | INDEX
| | | | | | | | INT=10
| | | | | | | | ELEMENT_TYPE
| | | | | | | | (int)
| | | | | | N_STMT_LIST (0,0)
| | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | N_EXP_ASSIGN (1b832ac0,0)
| | | | | | | | N_EXP_ARRAY (1b832ac0,1)
| | | | | | | | | N_EXP_IDENT (1b838260,0)
| | | | | | | | | | (ID="a") TYPE:1b838260 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_EXP_INT_CONST (1b832ac0,0)
| | | | | | | | | | | INT=3
| | | | | | | | | | N_EXP_CAST (1b832ac0,0)
| | | | | | | | | | | (int)
| | | | | | | | | | N_EXP_CHAR_CONST (1b832be0,0)
| | | | | | | | | | | INT=99
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_FUNCTION_CALL (1b832c70,0)
| | | | | | | | | | N_EXP_AMP (1b838840,0)
| | | | | | | | | | | N_EXP_IDENT (1b832d90,0)
| | | | | | | | | | | (ID="printf") TYPE:1b832d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | | N_EXP_STRING_LITERAL (1b832cb0,0)
| | | | | | | | | | | | LITERAL: "%c\n"
| | | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | | | N_EXP_ARRAY (1b832ac0,1)
| | | | | | | | | | | | | N_EXP_IDENT (1b838260,0)
| | | | | | | | | | | | | | (ID="a") TYPE:1b838260 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | | | | N_EXP_INT_CONST (1b832ac0,0)
| | | | | | | | | | | | | | | INT=3
| | | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | N_STMT_LIST_NIL (0,0)
start code generation
end code generation
```

```
-virtual-machine:~$ cat a.asm
```

```
    INT 0, 20
    SUP 0, main
    RET 0, 0
main:
    INT 0, 52
    LDA 1, 12
    LITI 0, 3
    LITI 0, 4
    MULI 0, 0
    OFFSET 0, 0
    LITI 0, 99
    STX 0, 0
    POP 0, 1
    INT 0, 12
    LDA 0, 12
    LDA 1, 12
    LITI 0, 3
    LITI 0, 4
    MULI 0, 0
    OFFSET 0, 0
    LDI 0, 0
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
    RET 0, 0
.literal 12 "%c\n"
```

```
-virtual-machine:~$ ./interpreter a.asm
```

```
===== symbol =====
```

```
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
```

```
===== code =====
```

```
0: INT 0,20
1: SUP 0,3
2: RET 0,0
3: INT 0,52
4: LDA 1,12
5: LITI 0,3
6: LITI 0,4
7: MULI 0,0
8: OFFSET 0,0
9: LITI 0,99
10: STX 0,0
11: POP 0,1
12: INT 0,12
13: LDA 0,12
14: LDA 1,12
15: LITI 0,3
16: LITI 0,4
17: MULI 0,0
18: OFFSET 0,0
```

```

18: OFFSET  0,0
19: LDI      0,0
20: POP      0,5
21: ADDR     0,-1
22: CAL      0,0
23: RET      0,0

```

start execution

c

end execution

[설명]

초기 프로그램이 실행될 때, 12+8(STRING 리터럴의 4의 배수로 align된 값)으로 INT가 되고 main 함수를 실행하는 것을 확인할 수 있다. 시멘틱 트리의 N_STMT_COMPOUND에서 알 수 있듯이 main 함수의 지역변수(int a[10])는 (int(4) * 배열의 크기(10))으로 총 12+40이 되어 52의 공간을 확보한다. a[3]의 값을 ASSIGN 연산을 통해 3이라는 값을 주게 되는데, 이때 a[3]은 a의 시작 주소에 '4(int_size)*3(인덱스)' 값만큼의 OFFSET을 설정하여 해당 위치에 'c'의 값을 주고 저장하게 된다. 이후 printf 함수를 처리하기 위해 12 만큼의 공간을 확보하고, 필요한 인자들(출력 문자열, a[3]의 값)을 스택으로 가져오고 함수를 호출한다. 모든 함수들은 종료 후에 리턴(RET)한다. interpreter의 결과를 보면 printf가 main은 symbol에 따라 숫자로 표현되고 a.asm과 동일한 결과를 출력한다. int와 char의 ASSIGN 연산에서 cast가 이루어지고 있는 것을 확인할 수 있다. 또한, printf를 통해 출력하고자 하는 'c'가 제대로 출력되는 것을 확인할 수 있다.

3.

[입력값]

```

-virtual-machine:~$ cat ex.c

float hamsoo(int a){
    return a;
}

int main(){
    hamsoo(2);
}

```


[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
start semantic analysis
--- warning at line 2:incompatible types in argument or return expr
===== semantic tree =====
N_PROGRAM (0,12)
| (ID="hansoo") TYPE:45fe1e0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:45f8ac0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | | (int)
| | | | | | | TYPE
| | | | | | | (float)
| | | | | | | BODY
| | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_RETURN (0,0)
| | | | | | | | N_EXP_CAST (45f8b50,0)
| | | | | | | | | (float)
| | | | | | | | | N_EXP_IDENT (45f8ac0,1)
| | | | | | | | | | (ID="a") TYPE:45f8ac0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_STMT_LIST_NIL (0,0)
| (ID="main") TYPE:45fe3f0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | | (int)
| | | | | | BODY
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | N_EXP_FUNCTION_CALL (45f8b50,0)
| | | | | | | | N_EXP_AMP (45fe6f0,0)
| | | | | | | | | N_EXP_IDENT (45fe1e0,0)
| | | | | | | | | | (ID="hansoo") TYPE:45fe1e0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | N_EXP_INT_CONST (45f8ac0,0)
| | | | | | | | | | | INT=2
| | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | N_STMT_LIST_NIL (0,0)
start code generation
end code generation
```



```

-virtual-machine:~$ cat a.asm

        INT 0, 12
        SUP 0, main
        RET 0, 0

hamsoo:

        INT 0, 16
        LDA 1, -4
        LOD 1, 12
        CVTF 0, 0
        STO 0, 0
        RET 0, 0
        RET 0, 0

main:

        INT 0, 12
        INT 0, 16
        LITI 0, 2
        POP 0, 4
        ADDR 0, hamsoo
        CAL 0, 0
        POP 0, 1
        RET 0, 0

```

```

-virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 10
5: hamsoo 3
===== code =====
0: INT 0,12
1: SUP 0,10
2: RET 0,0
3: INT 0,16
4: LDA 1,-4
5: LOD 1,12
6: CVTF 0,0
7: STO 0,0
8: RET 0,0
9: RET 0,0
10: INT 0,12
11: INT 0,16
12: LITI 0,2
13: POP 0,4
14: ADDR 0,3
15: CAL 0,0
16: POP 0,1
17: RET 0,0
start execution
end execution

```

[설명]

시멘틱 트리에서 보이는 N_PROGRAM의 value 값만큼 초반에 INT되는 것을 확인할 수 있다. main 함수는 파라미터나 지역변수가 없으므로 12이고, hamsoo는 파라미터를 1개 가지므로 총 16(12+4)의 크기를 확보한다. return value는 새로운 activation record가 생성된 바로 밑에 존재하므로 해당 레벨의 -4 주소를 가져와 값을 저장한다. 이 때 리턴 값을 int 타입으로, return의 타입인 float과 일치하지 않지만 형변환을 통해 float으로 변환된다. 어셈블

리 코드에서는 CVTF(float으로 형변환)를 사용하고 저장(STO)하는 것을 확인할 수 있다. 또한, 함수가 종료되면서 return(RET)이 된다. interpreter에서 확인할 수 있듯이, main과 hamsoo같은 symbol이 숫자 값으로 인식되고 출력되는 것을 확인할 수 있다.

4.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(){
    char a;
    char b;
    a='a'; a++;
    b='a'; b--;
    printf("%d %d\n", a, b);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```

N_PROGRAM (0,24)
| (ID="main") TYPE:f916f150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,8)
| | | | | | (ID="a") TYPE:f9169be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | TYPE
| | | | | | | (char 1)
| | | | | | (ID="b") TYPE:f9169be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | TYPE
| | | | | | | (char 1)
| | | | | N_STMT_LIST (0,0)
| | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | N_EXP_ASSIGN (f9169be0,0)
| | | | | | | | N_EXP_IDENT (f9169be0,1)
| | | | | | | | | (ID="a") TYPE:f9169be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_EXP_CHAR_CONST (f9169be0,0)
| | | | | | | | | | INT=97
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_POST_INC (f9169be0,0)
| | | | | | | | | N_EXP_IDENT (f9169be0,1)
| | | | | | | | | | (ID="a") TYPE:f9169be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (f9169be0,0)
| | | | | | | | | | N_EXP_IDENT (f9169be0,1)
| | | | | | | | | | | (ID="b") TYPE:f9169be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | N_EXP_CHAR_CONST (f9169be0,0)
| | | | | | | | | | | INT=97
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_POST_DEC (f9169be0,0)
| | | | | | | | | | N_EXP_IDENT (f9169be0,1)
| | | | | | | | | | | (ID="b") TYPE:f9169be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_FUNCTION_CALL (f9169c70,0)
| | | | | | | | | | N_EXP_AMP (f916fad0,0)
| | | | | | | | | | N_EXP_IDENT (f9169d90,0)
| | | | | | | | | | | (ID="printf") TYPE:f9169d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | N_ARG_LIST (0,12)
| | | | | | | | | | | N_EXP_STRING_LITERAL (f9169cb0,0)
| | | | | | | | | | | | LITERAL: "%d %d\n"

```

```

| | | | | | | | | | N_EXP_STRING_LITERAL (f9169cb0,0)
| | | | | | | | | | | LITERAL: "%d %d\n"
| | | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | | N_EXP_IDENT (f9169be0,1)
| | | | | | | | | | | | (ID="a") TYPE:f9169be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | | | N_EXP_IDENT (f9169be0,1)
| | | | | | | | | | | | | (ID="b") TYPE:f9169be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | | | | N_STMT_LIST_NIL (0,0)

```

start code generation

end code generation

```
-virtual-machine:~$ cat a.asm
    INT 0, 24
    SUP 0, main
    RET 0, 0
main:
    INT 0, 20
    LDA 1, 12
    LITI 0, 97
    STXB 0, 0
    POP 0, 1
    LOD 1, 12
    LDA 1, 12
    LDXB 0, 0
    INCI 0, 0
    STOB 0, 0
    POP 0, 1
    LDA 1, 16
    LITI 0, 97
    STXB 0, 0
    POP 0, 1
    LOD 1, 16
    LDA 1, 16
    LDXB 0, 0
    DECI 0, 0
    STOB 0, 0
    POP 0, 1
    INT 0, 12
    LDA 0, 12
    LOD 1, 12
    LOD 1, 16
    POP 0, 6
    ADDR 0, printf
    CAL 0, 0
    RET 0, 0
.literal 12 "%d %d\n"
```

```
virtual-machine: $ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,24
1: SUP 0,3
2: RET 0,0
3: INT 0,20
4: LDA 1,12
5: LITI 0,97
6: STXB 0,0
7: POP 0,1
8: LOD 1,12
9: LDA 1,12
10: LDXB 0,0
11: INCI 0,0
12: STOB 0,0
13: POP 0,1
14: LDA 1,16
15: LITI 0,97
16: STXB 0,0
17: POP 0,1
18: LOD 1,16
19: LDA 1,16
20: LDXB 0,0
21: DECI 0,0
22: STOB 0,0
23: POP 0,1
24: INT 0,12
25: LDA 0,12
26: LOD 1,12
27: LOD 1,16
28: POP 0,6
29: ADDR 0,-1
30: CAL 0,0
31: RET 0,0
start execution
98 96
end execution
```

[설명]

literal의 align된 값(12)에 12를 더해 총 24만큼 INT가 되어 프로그램이 시작되고, main 함수에서는 2개의 지역변수가 존재해 20만큼 INT된다. 후위 증가, 후위 감소 연산은 char 타입이 int 타입으로 취급되어 각각 INCI, DECI이 사용된다. 이 때 char는 1바이트이므로 STOB, LDXB가 사용되는 것을 확인할 수 있다. 또한, 변수 a와 b의 값으로 'a' 값을 ASSIGN 연산하였는데, 'a'의 아스키코드 값(97)에서 1 증가, 1 감소한 결과를 확인할 수 있다.

5.

[입력값]

```
-virtual-machine:~$ cat ex.c

int main(){
    int a;
    char b;

    a=55; ++a;
    b='a'; --b;
    printf("%d %d\n", a, b);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
===== semantic tree =====
N_PROGRAM (0,24)
| (ID="main") TYPE:d3b9f150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,8)
| | | | | | (ID="a") TYPE:d3b99ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | TYPE
| | | | | | | (int)
| | | | | | (ID="b") TYPE:d3b99be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | TYPE
| | | | | | | (char 1)
| | | | | N_STMT_LIST (0,0)
| | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | N_EXP_ASSIGN (d3b99ac0,0)
| | | | | | | | N_EXP_IDENT (d3b99ac0,1)
| | | | | | | | | (ID="a") TYPE:d3b99ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_EXP_INT_CONST (d3b99ac0,0)
| | | | | | | | | | INT=55
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_PRE_INC (d3b99ac0,0)
| | | | | | | | | N_EXP_IDENT (d3b99ac0,1)
| | | | | | | | | | (ID="a") TYPE:d3b99ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (d3b99be0,0)
| | | | | | | | | | N_EXP_IDENT (d3b99be0,1)
| | | | | | | | | | (ID="b") TYPE:d3b99be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | N_EXP_CHAR_CONST (d3b99be0,0)
| | | | | | | | | | | INT=97
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_PRE_DEC (d3b99be0,0)
| | | | | | | | | | N_EXP_IDENT (d3b99be0,1)
| | | | | | | | | | (ID="b") TYPE:d3b99be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_FUNCTION_CALL (d3b99c70,0)
| | | | | | | | | | | N_EXP_AMP (d3b99fad0,0)
| | | | | | | | | | | | N_EXP_IDENT (d3b99d90,0)
| | | | | | | | | | | | | (ID="printf") TYPE:d3b99d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | | | N_ARG_LIST (0,12)
| | | | | | | | | | | | | | N_EXP_STRING_LITERAL (d3b99cb0,0)
| | | | | | | | | | | | | | | LITERAL: "%d %d\n"
| | | | | | | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | | | | | | N_EXP_IDENT (d3b99ac0,1)
| | | | | | | | | | | | | | | | (ID="a") TYPE:d3b99ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | | | | | | | | N_EXP_IDENT (d3b99be0,1)
| | | | | | | | | | | | | | | | | | (ID="b") TYPE:d3b99be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)

start code generation
end code generation
```



```

-virtual-machine: $ cat a.asm
    INT 0, 24
    SUP 0, main
    RET 0, 0
main:
    INT 0, 20
    LDA 1, 12
    LITI 0, 55
    STX 0, 0
    POP 0, 1
    LDA 1, 12
    LDX 0, 1
    INCI 0, 0
    STX 0, 0
    POP 0, 1
    LDA 1, 16
    LITI 0, 97
    STXB 0, 0
    POP 0, 1
    LDA 1, 16
    LDXB 0, 0
    DECI 0, 0
    STXB 0, 0
    POP 0, 1
    INT 0, 12
    LDA 0, 12
    LOD 1, 12
    LOD 1, 16
    POP 0, 6
    ADDR 0, printf
    CAL 0, 0
    RET 0, 0
.literal 12 "%d %d\n"

```

```

-virtual-machine: $ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,24
1: SUP 0,3
2: RET 0,0
3: INT 0,20
4: LDA 1,12
5: LITI 0,55
6: STX 0,0
7: POP 0,1
8: LDA 1,12
9: LDX 0,1
10: INCI 0,0
11: STX 0,0
12: POP 0,1
13: LDA 1,16
14: LITI 0,97
15: STXB 0,0
16: POP 0,1
17: LDA 1,16

```



```

17: LDA    1,16
18: LDXB   0,0
19: DECI   0,0
20: STXB   0,0
21: POP    0,1
22: INT    0,12
23: LDA    0,12
24: LOD    1,12
25: LOD    1,16
26: POP    0,6
27: ADDR   0,-1
28: CAL    0,0
29: RET    0,0

```

start execution

56 96

end execution

[설명]

이전과 결과와 동일하게 활성화 레코드의 크기를 설정하였다. 그리고 int 타입인 a는 LDX, STXB를 사용하였고 1바이트인 char 타입 b는 LDXB, STXB를 사용하였다. 그리고 두 지역 변수 모두 동일하게 int 타입으로 취급되어 LITI, INCI, DECI가 사용되는 것을 확인할 수 있다. 계산 결과 또한 적절한 값이 출력된다.

6.

[입력값]

```

virtual-machine:~$ cat ex.c
int main(){
    float a;
    scanf("%f",&a);
    a=!a;
    printf("%d\n",a);
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```

start semantic analysis
--- warning at line 4:incompatible types in assignment expression
===== semantic tree =====
N_PROGRAM (0,28)
| (ID="main") TYPE:9427a150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,4)
| | | | | | (ID="a") TYPE:94274b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | (float)
| | | | | N_STMT_LIST (0,0)
| | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | N_EXP_FUNCTION_CALL (94274c70,0)
| | | | | | | N_EXP_AMP (9427a960,0)
| | | | | | | N_EXP_IDENT (94274ec0,0)
| | | | | | | | (ID="scanf") TYPE:94274ec0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | N_ARG_LIST (0,8)
| | | | | | | | N_EXP_STRING_LITERAL (94274cb0,0)
| | | | | | | | LITERAL: "%f"

```

```

| | | | | | | | | | LITERAL: "%f"
| | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | N_EXP_AMP (9427a9e0,0)
| | | | | | | | | | N_EXP_IDENT (94274b50,1)
| | | | | | | | | | (ID="a") TYPE:94274b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_ASSIGN (94274b50,0)
| | | | | | | | | | N_EXP_IDENT (94274b50,1)
| | | | | | | | | | (ID="a") TYPE:94274b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_EXP_CAST (94274b50,0)
| | | | | | | | | | (float)
| | | | | | | | | | N_EXP_NOT (94274ac0,0)
| | | | | | | | | | N_EXP_IDENT (94274b50,1)
| | | | | | | | | | (ID="a") TYPE:94274b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_FUNCTION_CALL (94274c70,0)
| | | | | | | | | | N_EXP_AMP (9427aa60,0)
| | | | | | | | | | N_EXP_IDENT (94274d90,0)
| | | | | | | | | | (ID="printf") TYPE:94274d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | N_EXP_STRING_LITERAL (94274cb0,0)
| | | | | | | | | | LITERAL: "%d\n"
| | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | N_EXP_IDENT (94274b50,1)
| | | | | | | | | | (ID="a") TYPE:94274b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | | | | N_STMT_LIST_NIL (0,0)
start code generation
end code generation

```

```

-virtual-machine:-$ cat a.asm

```

```

        INT    0, 28
        SUP    0, main
        RET    0, 0

main:

        INT    0, 16
        INT    0, 12
        LDA    0, 12
        LDA    1, 12
        POP    0, 5
        ADDR   0, scanf
        CAL    0, 0
        LDA    1, 12
        LOD    1, 12
        NOT    0, 0
        CVTF   0, 0
        STX    0, 0
        POP    0, 1
        INT    0, 12
        LDA    0, 20
        LOD    1, 12
        POP    0, 5
        ADDR   0, printf
        CAL    0, 0
        RET    0, 0

.literal 12  "%f"
.literal 20  "%d\n"

```

```
virtual-machine:~$ ./interpreter a.asm
===== symbol =====
 1: printf  -1
 2: malloc  -2
 3: scanf   -3
 4: main     3
===== code =====
 0: INT      0,28
 1: SUP      0,3
 2: RET      0,0
 3: INT      0,16
 4: INT      0,12
 5: LDA      0,12
 6: LDA      1,12
 7: POP      0,5
 8: ADDR     0,-3
 9: CAL      0,0
10: LDA      1,12
11: LOD      1,12
12: NOT      0,0
13: CVTF     0,0
14: STX      0,0
15: POP      0,1
16: INT      0,12
17: LDA      0,20
18: LOD      1,12
19: POP      0,5
20: ADDR     0,-1
21: CAL      0,0
22: RET      0,0
start execution
2.25
0
end execution
```

[설명]

scanf, printf에서의 문자열이 literal table이 입력되고 이로 인해 28만큼의 크기를 확보한다. 또한 main 함수에서는 지역 변수가 1개이기 때문에 16(12+4)만큼 확보하는 것을 확인할 수 있다. 또한, NOT 연산을 하고 float으로 형변환(CVTF)이 되는 것을 시멘틱 트리를 통해 확인할 수 있다. 2.25는 참이 때문에 NOT 연산을 하면 0이 출력된다. %d로 출력하기 때문에 형변환이 된 것이지만, 해당 부분은 시멘틱 트리에 출력되지 않는 부분이다. 또한, 함수를 호출할 때 POP 0,5에서 5는 스택으로 올라온 static_link, dynamic_link, return_address, 스트링 리터럴, 지역 변수, 총 5개를 제거(pop)하기 위함이다.

7.

[입력값]

```
virtual-machine: $ cat ex.c
int main(){
    int a;
    a=2;
    a+=a;
    a-=a;
    printf("%d\n",a);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_PROGRAM (0,20)
| (ID="main") TYPE:aa6db150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,4)
| | | | | | (ID="a") TYPE:aa6d5ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | | (int)
| | | | | N_STMT_LIST (0,0)
| | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | N_EXP_ASSIGN (aa6d5ac0,0)
| | | | | | | | N_EXP_IDENT (aa6d5ac0,1)
| | | | | | | | | (ID="a") TYPE:aa6d5ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_EXP_INT_CONST (aa6d5ac0,0)
| | | | | | | | | INT=2
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (aa6d5ac0,0)
| | | | | | | | | N_EXP_IDENT (aa6d5ac0,1)
| | | | | | | | | | (ID="a") TYPE:aa6d5ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_EXP_PLUS (aa6d5ac0,0)
| | | | | | | | | | N_EXP_IDENT (aa6d5ac0,1)
| | | | | | | | | | | (ID="a") TYPE:aa6d5ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (aa6d5ac0,0)
| | | | | | | | | | N_EXP_IDENT (aa6d5ac0,1)
| | | | | | | | | | | (ID="a") TYPE:aa6d5ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_EXP_MINUS (aa6d5ac0,0)
| | | | | | | | | | | N_EXP_IDENT (aa6d5ac0,1)
| | | | | | | | | | | | (ID="a") TYPE:aa6d5ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_FUNCTION_CALL (aa6d5c70,0)
| | | | | | | | | | | N_EXP_AMP (aa6db980,0)
| | | | | | | | | | | | N_EXP_IDENT (aa6d5d90,0)
| | | | | | | | | | | | | (ID="printf") TYPE:aa6d5d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | | | N_EXP_STRING_LITERAL (aa6d5cb0,0)
| | | | | | | | | | | | | LITERAL: "%d\n"
| | | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | | | | N_EXP_IDENT (aa6d5ac0,1)
| | | | | | | | | | | | | | (ID="a") TYPE:aa6d5ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | N_STMT_LIST_NIL (0,0)
start code generation
end code generation
```

```

-virtual-machine:~$ cat a.asm
        INT 0, 20
        SUP 0, main
        RET 0, 0
main:
        INT 0, 16
        LDA 1, 12
        LITI 0, 2
        STX 0, 0
        POP 0, 1
        LDA 1, 12
        LOD 1, 12
        STX 0, 0
        POP 0, 1
        LDA 1, 12
        LOD 1, 12
        MINUSI 0, 0
        STX 0, 0
        POP 0, 1
        INT 0, 12
        LDA 0, 12
        LOD 1, 12
        POP 0, 5
        ADDR 0, printf
        CAL 0, 0
        RET 0, 0
.literal 12 "%d\n"

```

```

-virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,20
1: SUP 0,3
2: RET 0,0
3: INT 0,16
4: LDA 1,12
5: LITI 0,2
6: STX 0,0
7: POP 0,1
8: LDA 1,12
9: LOD 1,12
10: STX 0,0
11: POP 0,1
12: LDA 1,12
13: LOD 1,12
14: MINUSI 0,0
15: STX 0,0
16: POP 0,1
17: INT 0,12
18: LDA 0,12
19: LOD 1,12
20: POP 0,5
21: ADDR 0,-1
22: CAL 0,0
23: RET 0,0
start execution
-2
end execution

```


[설명]

PLUS 연산인 경우 별다른 작업 없이 기존의 값을 전달(LOD)하는 것을 확인할 수 있고, MINUS인 경우 int 타입이므로, MINUS를 통해 변수에 빼기 작업을 하는 것을 확인할 수 있다.

8.

[입력값]

```
virtual-machine:~$ cat ex.c
int main(){
    char *ch1;
    char ch2;
    ch2='a';
    ch1=&ch2;
    ch2=*ch1;
    printf("%c %c\n",*ch1,ch2);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
===== semantic tree =====
N_PROGRAM (0,24)
| (ID="main") TYPE:5ebc3150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,8)
| | | | | | (ID="ch1") TYPE:5ebc31d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | TYPE
| | | | | | | | POINTER
| | | | | | | | | ELEMENT_TYPE
| | | | | | | | | (char 1)
| | | | | | | (ID="ch2") TYPE:5ebbdbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | TYPE
| | | | | | | | (char 1)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (5ebbdbe0,0)
| | | | | | | | | N_EXP_IDENT (5ebbdbe0,1)
| | | | | | | | | | (ID="ch2") TYPE:5ebbdbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | N_EXP_CHAR_CONST (5ebbdbe0,0)
| | | | | | | | | | | INT=97
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (5ebc31d0,0)
| | | | | | | | | | N_EXP_IDENT (5ebc31d0,1)
| | | | | | | | | | | (ID="ch1") TYPE:5ebc31d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_EXP_AMP (5ebc3b10,0)
| | | | | | | | | | | N_EXP_IDENT (5ebbdbe0,1)
| | | | | | | | | | | (ID="ch2") TYPE:5ebbdbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (5ebbdbe0,0)
| | | | | | | | | | N_EXP_IDENT (5ebbdbe0,1)
| | | | | | | | | | | (ID="ch2") TYPE:5ebbdbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | N_EXP_STAR (5ebbdbe0,1)
| | | | | | | | | | | N_EXP_IDENT (5ebc31d0,1)
| | | | | | | | | | | | (ID="ch1") TYPE:5ebc31d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_FUNCTION_CALL (5ebbd70,0)
| | | | | | | | | | N_EXP_AMP (5ebc3b50,0)
| | | | | | | | | | | N_EXP_IDENT (5ebbdd90,0)
| | | | | | | | | | | | (ID="printf") TYPE:5ebbdd90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | N_ARG_LIST (0,12)
| | | | | | | | | | | N_EXP_STRING_LITERAL (5ebbdcb0,0)
| | | | | | | | | | | | LITERAL: "%c %c\n"
| | | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | | N_EXP_STAR (5ebbdbe0,1)
| | | | | | | | | | | | N_EXP_IDENT (5ebc31d0,1)
| | | | | | | | | | | | (ID="ch1") TYPE:5ebc31d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
```



```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
 1: printf  -1
 2: malloc  -2
 3: scanf   -3
 4: main     3
===== code =====
 0: INT      0,24
 1: SUP      0,3
 2: RET      0,0
 3: INT      0,20
 4: LDA      1,16
 5: LITI     0,97
 6: STXB     0,0
 7: POP      0,1
 8: LDA      1,12
 9: LDA      1,16
10: STX      0,0
11: POP      0,1
12: LDA      1,16
13: LOD      1,12
14: LDIB     0,0
15: STXB     0,0
16: POP      0,1
17: INT      0,12
18: LDA      0,12
19: LOD      1,12
20: LDIB     0,0
21: LOD      1,16
22: POP      0,6
23: ADDR     0,-1
24: CAL      0,0
25: RET      0,0
start execution
a a
end execution

```

[설명]

AMP와 STAR 연산을 통해 각각 변수의 주소와 값을 가져온다. 시멘틱 트리를 통해 결과를 가져올 대상의 변수를 노드에 연결한 것을 확인할 수 있다. AMP에서는 `gen_expression_left`를 통해 `N_EXP_AMP` 하위에 있는 identifier `ch2`의 주소를 가져와서 저장하는 것을 확인할 수 있다. 그리고 STAR 연산은 포인터 타입의 `ch1`의 값을 가져오는 것을 확인할 수 있다. 이때 값은 1바이트이므로 `LDIB`, `STXB`가 사용된다.

9.

[입력값]

```

virtual-machine:~$ cat ex.c
int main(){
    int a[5];
    printf("%d ",sizeof(int));
    printf("%d\n",sizeof(a));
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_PROGRAM (0,28)
| (ID="main") TYPE:52464150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,20)
| | | | | | (ID="a") TYPE:52464260 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | TYPE
| | | | | | | | ARRAY
| | | | | | | | INDEX
| | | | | | | | | INT=5
| | | | | | | | | ELEMENT_TYPE
| | | | | | | | | (int)
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_FUNCTION_CALL (5245ec70,0)
| | | | | | | | | | N_EXP_AMP (52464880,0)
| | | | | | | | | | N_EXP_IDENT (5245ed90,0)
| | | | | | | | | | | (ID="printf") TYPE:5245ed90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | N_EXP_STRING_LITERAL (5245ecb0,0)
| | | | | | | | | | | LITERAL: "%d "
| | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | | N_EXP_SIZE_TYPE (5245eac0,0)
| | | | | | | | | | | | INT=4
| | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_FUNCTION_CALL (5245ec70,0)
| | | | | | | | | | N_EXP_AMP (52464900,0)
| | | | | | | | | | N_EXP_IDENT (5245ed90,0)
| | | | | | | | | | | (ID="printf") TYPE:5245ed90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | N_EXP_STRING_LITERAL (5245ecb0,0)
| | | | | | | | | | | LITERAL: "%d\n"
| | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | | N_EXP_SIZE_EXP (5245eac0,0)
| | | | | | | | | | | | INT=20
| | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | N_STMT_LIST_NIL (0,0)
```

```
-virtual-machine: $ cat a.asm

    INT 0, 28
    SUP 0, main
    RET 0, 0

main:

    INT 0, 32
    INT 0, 12
    LDA 0, 12
    LITI 0, 4
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
    INT 0, 12
    LDA 0, 20
    LITI 0, 20
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
    RET 0, 0

.literal 12 "%d "
.literal 20 "%d\n"
```

```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,28
1: SUP 0,3
2: RET 0,0
3: INT 0,32
4: INT 0,12
5: LDA 0,12
6: LITI 0,4
7: POP 0,5
8: ADDR 0,-1
9: CAL 0,0
10: INT 0,12
11: LDA 0,20
12: LITI 0,20
13: POP 0,5
14: ADDR 0,-1
15: CAL 0,0
16: RET 0,0
start execution
4 20
end execution

```

[설명]

N_EXP_SIZE_TYPE은 노드의 가운데 자식 노드의 타입 크기를 LITI를 통해 바로 정수값으로 가져온다. N_EXP_SIZE_EXP 역시도 배열('a[5]')의 크기를 바로 정수값으로 바꾼 값을 LITI로 가져오는 것을 확인할 수 있다. interpreter에서도 크기가 각각 4, 20으로 적절한 값이 출력된 것을 확인할 수 있다.

10.

[입력값]

```

virtual-machine:~$ cat ex.c
int main(){
    int a;
    int b;
    int c;
    a=5;
    b=1;
    c=0;
    a*a;
    b/c;
    b%a;
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```

N_PROGRAM (0,12)
| (ID="main") TYPE:9065b150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | TYPE
| | | | | (int)
| | | | BODY

```

```

| | | | BODY
| | | | | N_STMT_COMPOUND (0,12)
| | | | | | (ID="a") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | | (int)
| | | | | | (ID="b") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | TYPE
| | | | | | | (int)
| | | | | | (ID="c") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
| | | | | | TYPE
| | | | | | | (int)
| | | | | N_STMT_LIST (0,0)
| | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | N_EXP_ASSIGN (90655ac0,0)
| | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | (ID="a") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | N_EXP_INT_CONST (90655ac0,0)
| | | | | | | | | INT=5
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (90655ac0,0)
| | | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | | (ID="b") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | N_EXP_INT_CONST (90655ac0,0)
| | | | | | | | | | INT=1
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (90655ac0,0)
| | | | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | | | (ID="c") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
| | | | | | | | | | N_EXP_INT_CONST (90655ac0,0)
| | | | | | | | | | | INT=0
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_MUL (90655ac0,0)
| | | | | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | | | | (ID="a") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | | | | (ID="a") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | N_EXP_DIV (90655ac0,0)
| | | | | | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | | | | | (ID="b") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | | | | | (ID="c") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
| | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | N_EXP_MOD (90655ac0,0)
| | | | | | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | | | | | (ID="b") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | | | N_EXP_IDENT (90655ac0,1)
| | | | | | | | | | | | | (ID="a") TYPE:90655ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_STMT_LIST_NIL (0,0)

```

start code generation

```

-vmachine:~$ cat a.asm

        INT    0, 12
        SUP    0, main
        RET    0, 0

main:

        INT    0, 24
        LDA    1, 12
        LITI   0, 5
        STX    0, 0
        POP    0, 1

```

```

        STX 0, 0
        POP 0, 1
        LDA 1, 16
        LITI 0, 1
        STX 0, 0
        POP 0, 1
        LDA 1, 20
        LITI 0, 0
        STX 0, 0
        POP 0, 1
        LOD 1, 12
        LOD 1, 12
        MULI 0, 0
        POP 0, 1
        LOD 1, 16
        LOD 1, 20
        DIVI 0, 0
        POP 0, 1
        LOD 1, 16
        LOD 1, 12
        MOD 0, 0
        POP 0, 1
        RET 0, 0

```

```

-virtual-machine: $ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,12
1: SUP 0,3
2: RET 0,0
3: INT 0,24
4: LDA 1,12
5: LITI 0,5
6: STX 0,0
7: POP 0,1
8: LDA 1,16
9: LITI 0,1
10: STX 0,0
11: POP 0,1
12: LDA 1,20
13: LITI 0,0
14: STX 0,0
15: POP 0,1
16: LOD 1,12
17: LOD 1,12
18: MULI 0,0
19: POP 0,1
20: LOD 1,16
21: LOD 1,20
22: DIVI 0,0
23: POP 0,1
24: LOD 1,16
25: LOD 1,12
26: MOD 0,0
27: POP 0,1
28: RET 0,0
start execution
runtime error at pc 23 : devide by zero

```

[설명]

연산의 결과를 저장할 lvalue가 없기 때문에 lvalue의 주소를 가져오는 작업(LDA)이 필요 없고 연산후에 저장(STO)하는 작업도 존재하지 않는다. 연산 후에는 연산을 수행한 stack의 top 부분만 pop하면 된다. 지역 변수 3개는 모두 int 타입으로 계산(MULI, DIVI)은 int형이다. 또한, interpreter의 결과를 확인했을 때 'devided by zero' error message가 출력되는 것을 확인할 수 있다. 이는 0의 값을 가진 c로 나누는 것을 시도했기 때문이다.

11.

[입력값]

```
-virtual-machine: $ cat ex.c
int main(){
    float a;
    float b;
    float c;
    a=5.0;
    b=1.0;
    c=0.0;
    a*a;
    b/c;
    printf("%f\n",a*a);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
-virtual-machine: $ cat a.asm
      INT 0, 32
      SUP 0, main
      RET 0, 0
main:
      INT 0, 24
      LDA 1, 12
      LOD 0, 12
      STX 0, 0
      POP 0, 1
      LDA 1, 16
      LOD 0, 16
      STX 0, 0
      POP 0, 1
      LDA 1, 20
      LOD 0, 20
      STX 0, 0
      POP 0, 1
      LOD 1, 12
      LOD 1, 12
      MULF 0, 0
      POP 0, 1
      LOD 1, 16
      LOD 1, 20
      DIVF 0, 0
      POP 0, 1
      INT 0, 12
      LDA 0, 24
      LOD 1, 12
      LOD 1, 12
      MULF 0, 0
      POP 0, 5
      ADDR 0, printf
      CAL 0, 0
      RET 0, 0
.literal 12 0.000000
.literal 16 0.000000
.literal 20 0.000000
.literal 24 "%f"
```



```

-virtual-machine: $ ./interpreter a.asm
===== symbol =====
  1: printf  -1
  2: malloc  -2
  3: scanf   -3
  4: main     3
===== code =====
  0: INT      0,32
  1: SUP      0,3
  2: RET      0,0
  3: INT      0,24
  4: LDA      1,12
  5: LOD      0,12
  6: STX      0,0
  7: POP      0,1
  8: LDA      1,16
  9: LOD      0,16
10: STX      0,0
11: POP      0,1
12: LDA      1,20
13: LOD      0,20
14: STX      0,0
15: POP      0,1
16: LOD      1,12
17: LOD      1,12
18: MULF     0,0
19: POP      0,1
20: LOD      1,16
21: LOD      1,20
22: DIVF     0,0
23: POP      0,1
24: INT      0,12
25: LDA      0,24
26: LOD      1,12
27: LOD      1,12
28: MULF     0,0
29: POP      0,5
30: ADDR     0,-1
31: CAL      0,0
32: RET      0,0
start execution
0.000000end execution

```

[설명]

트리의 구조는 이전(10번)에 printf 함수를 추가한 것 외에는 동일하게 출력된다. 리터럴 테이블의 개수는 정확하게 생성되지만 값이 제대로 저장되고 출력되지 않고 있다. 이전의 결과와는 다르게 ASSIGN 연산에서 LITI가 아닌 LOD로 리터럴 테이블에서 값을 읽어오는 것을 확인 할 수 있다.

12.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(){
    int a;
    int b;
    a=3;
    b=10;
    a+b;
    a-b;
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_PROGRAM (0,12)
| (ID="main") TYPE:a4e39150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,8)
| | | | | | (ID="a") TYPE:a4e33ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | TYPE
| | | | | | | | (int)
| | | | | | (ID="b") TYPE:a4e33ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | TYPE
| | | | | | | | (int)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (a4e33ac0,0)
| | | | | | | | | N_EXP_IDENT (a4e33ac0,1)
| | | | | | | | | | (ID="a") TYPE:a4e33ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_EXP_INT_CONST (a4e33ac0,0)
| | | | | | | | | | INT=3
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ASSIGN (a4e33ac0,0)
| | | | | | | | | | N_EXP_IDENT (a4e33ac0,1)
| | | | | | | | | | (ID="b") TYPE:a4e33ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | N_EXP_INT_CONST (a4e33ac0,0)
| | | | | | | | | | INT=10
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_ADD (a4e33ac0,0)
| | | | | | | | | | N_EXP_IDENT (a4e33ac0,1)
| | | | | | | | | | | (ID="a") TYPE:a4e33ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_EXP_IDENT (a4e33ac0,1)
| | | | | | | | | | | (ID="b") TYPE:a4e33ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | N_EXP_SUB (a4e33ac0,0)
| | | | | | | | | | | | N_EXP_IDENT (a4e33ac0,1)
| | | | | | | | | | | | | (ID="a") TYPE:a4e33ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | | N_EXP_IDENT (a4e33ac0,1)
| | | | | | | | | | | | | (ID="b") TYPE:a4e33ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | N_STMT_LIST_NIL (0,0)
```

```

virtual-machine:~$ cat a.asm
    INT 0, 12
    SUP 0, main
    RET 0, 0
main:
    INT 0, 20
    LDA 1, 12
    LITI 0, 3
    STX 0, 0
    POP 0, 1
    LDA 1, 16
    LITI 0, 10
    STX 0, 0
    POP 0, 1
    LOD 1, 12
    LOD 1, 16
    ADDI 0, 0
    POP 0, 1
    LOD 1, 12
    LOD 1, 16
    SUBI 0, 0
    POP 0, 1
    RET 0, 0
virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,12
1: SUP 0,3
2: RET 0,0
3: INT 0,20
4: LDA 1,12
5: LITI 0,3
6: STX 0,0
7: POP 0,1
8: LDA 1,16
9: LITI 0,10
10: STX 0,0
11: POP 0,1
12: LOD 1,12
13: LOD 1,16
14: ADDI 0,0
15: POP 0,1
16: LOD 1,12
17: LOD 1,16
18: SUBI 0,0
19: POP 0,1
20: RET 0,0
start execution
end execution

```

[설명]

두 변수 모두 int 타입이기 때문에 ADDI, SUBI를 통해 연산이 된다. ./compiler와 ./interpreter 모두 적절한 출력 결과를 출력하는 것을 확인할 수 있다

13.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(){
    int *p;
    p=p+2;
    p=p-2;
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_PROGRAM (0,12)
| (ID="main") TYPE:2ed8d150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,4)
| | | | | | (ID="p") TYPE:2ed8d1d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | TYPE
| | | | | | | | POINTER
| | | | | | | | | ELEMENT_TYPE
| | | | | | | | | (int)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (2ed8d1d0,0)
| | | | | | | | | N_EXP_IDENT (2ed8d1d0,1)
| | | | | | | | | | (ID="p") TYPE:2ed8d1d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_EXP_ADD (2ed8d1d0,0)
| | | | | | | | | | N_EXP_IDENT (2ed8d1d0,1)
| | | | | | | | | | | (ID="p") TYPE:2ed8d1d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_EXP_INT_CONST (2ed87ac0,0)
| | | | | | | | | | | INT=2
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (2ed8d1d0,0)
| | | | | | | | | N_EXP_IDENT (2ed8d1d0,1)
| | | | | | | | | | (ID="p") TYPE:2ed8d1d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_EXP_SUB (2ed8d1d0,0)
| | | | | | | | | | N_EXP_IDENT (2ed8d1d0,1)
| | | | | | | | | | | (ID="p") TYPE:2ed8d1d0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | N_EXP_INT_CONST (2ed87ac0,0)
| | | | | | | | | | | INT=2
| | | | | | N_STMT_LIST_NIL (0,0)
```

```

virtual-machine: $ cat a.asm
main:
    INT 0, 12
    SUP 0, main
    RET 0, 0

    INT 0, 16
    LDA 1, 12
    LOD 1, 12
    LITI 0, 2
    LITI 0, 4
    MULI 0, 0
    ADDI 0, 0
    STX 0, 0
    POP 0, 1
    LDA 1, 12
    LOD 1, 12
    LITI 0, 2
    LITI 0, 4
    MULI 0, 0
    SUBI 0, 0
    STX 0, 0
    POP 0, 1
    RET 0, 0

virtual-machine: $ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,12
1: SUP 0,3
2: RET 0,0
3: INT 0,16
4: LDA 1,12
5: LOD 1,12
6: LITI 0,2
7: LITI 0,4
8: MULI 0,0
9: ADDI 0,0
10: STX 0,0
11: POP 0,1
12: LDA 1,12
13: LOD 1,12
14: LITI 0,2
15: LITI 0,4
16: MULI 0,0
17: SUBI 0,0
18: STX 0,0
19: POP 0,1
20: RET 0,0
start execution
end execution

```

[설명]

왼쪽 수식이나 오른쪽 수식에 pointer 타입이나 배열 타입이 오면 ADDI/ADDF 대신 LITI와 MULI가 수행된다. 주소 연산이기 때문에 사실상 offset을 구해주어야하는 것이다. 이때 더해지거나 빼지는 피연산자 중 정수 상수 값은 바로 LITI 수행으로 값을 가져온다. 시멘틱 트리와 어셈블리어, interpreter 결과가 적절하게 나오는 것을 확인할 수 있다.

14.

[입력값]

```

-virtual-machine:~$ cat ex.c
int main(){
    float a,b,c;
    a=2; b=3; c=4;
    a<b;
    b>c;
    a<=c;
    b>=a;
}

```


[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_PROGRAM (0,12)
| (ID="main") TYPE:c4fdf150 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | TYPE
| | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,12)
| | | | | | (ID="a") TYPE:c4fd9b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | | (float)
| | | | | | (ID="b") TYPE:c4fd9b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | TYPE
| | | | | | | (float)
| | | | | | (ID="c") TYPE:c4fd9b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
| | | | | | TYPE
| | | | | | | (float)
| | | | | N_STMT_LIST (0,0)
```

```

| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (c4fd9b50,0)
| | | | | | | | | N_EXP_IDENT (c4fd9b50,1)
| | | | | | | | | | (ID="c") TYPE:c4fd9b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
| | | | | | | | | N_EXP_CAST (c4fd9b50,0)
| | | | | | | | | | (float)
| | | | | | | | | N_EXP_INT_CONST (c4fd9ac0,0)
| | | | | | | | | | INT=4
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | N_EXP_LSS (c4fd9ac0,0)
| | | | | | | | | | N_EXP_IDENT (c4fd9b50,1)
| | | | | | | | | | | (ID="a") TYPE:c4fd9b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | N_EXP_IDENT (c4fd9b50,1)
| | | | | | | | | | | (ID="b") TYPE:c4fd9b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | N_STMT_LIST (0,0)
```

```

--virtual-machine: $ cat a.asm

    INT    0, 12
    SUP    0, main
    RET    0, 0

main:

    INT    0, 24
    LDA    1, 12
    LITI   0, 2
    CVTF   0, 0
    STX    0, 0
    POP    0, 1
    LDA    1, 16
    LITI   0, 3
    CVTF   0, 0
    STX    0, 0
    POP    0, 1
    LDA    1, 20
    LITI   0, 4
    CVTF   0, 0
    STX    0, 0
    POP    0, 1
    LOD    1, 12
    LOD    1, 16
    LSSF   0, 0
    POP    0, 1
    LOD    1, 16
    LOD    1, 20
    GTRF   0, 0
    POP    0, 1
    LOD    1, 12
    LOD    1, 20
    LEQF   0, 0
    POP    0, 1
    LOD    1, 16
    LOD    1, 12
    GEQF   0, 0
    POP    0, 1
    RET    0, 0
```


[설명]

시멘틱 트리의 일부를 가져왔으며, 세 개의 변수 모두 float 타입이기 때문에 비교 연산자 (LSSF, GTRF, LEQF, GEQF)가 사용된다. 또한, float형 타입으로 선언하였는데 int형 값을 ASSIGN 연산하여 cast 연산(CVTF)이 수행된다. 모두 적절한 결과를 보여준다.

15.

[입력값]

```
virtual-machine:~$ cat ex.c
int main(){
    float a,b;
    char c;
    a=2.0; b=3.0;
    c='c';
    printf("%d ",a==b);
    printf("%d ",a!=b);
    printf("%d", a==c);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```

| | | N_STMT_EXPRESSION (0,0)
| | | | N_EXP_FUNCTION_CALL (5ef87c70,0)
| | | | | N_EXP_AMP (5ef8e0c0,0)
| | | | | | N_EXP_IDENT (5ef87d90,0)
| | | | | | | (ID="printf") TYPE:5ef87d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | N_ARG_LIST (0,8)
| | | | | | N_EXP_STRING_LITERAL (5ef87cb0,0)
| | | | | | | LITERAL: "%d "
| | | | | | N_ARG_LIST (0,4)
| | | | | | | N_EXP_EQL (5ef87ac0,0)
| | | | | | | | N_EXP_IDENT (5ef87b50,1)
| | | | | | | | | (ID="a") TYPE:5ef87b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | N_EXP_IDENT (5ef87b50,1)
| | | | | | | | | (ID="b") TYPE:5ef87b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | N_ARG_LIST_NIL (0,0)
| | | | N_STMT_LIST (0,0)
| | | | | N_STMT_EXPRESSION (0,0)
| | | | | | N_EXP_FUNCTION_CALL (5ef87c70,0)
| | | | | | | N_EXP_AMP (5ef8e140,0)
| | | | | | | | N_EXP_IDENT (5ef87d90,0)
| | | | | | | | | (ID="printf") TYPE:5ef87d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | N_EXP_STRING_LITERAL (5ef87cb0,0)
| | | | | | | | | | LITERAL: "%d "
| | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | N_EXP_NEQ (5ef87ac0,0)
| | | | | | | | | | | N_EXP_IDENT (5ef87b50,1)
| | | | | | | | | | | | (ID="a") TYPE:5ef87b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | N_EXP_IDENT (5ef87b50,1)
| | | | | | | | | | | | (ID="b") TYPE:5ef87b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_FUNCTION_CALL (5ef87c70,0)
| | | | | | | | | N_EXP_AMP (5ef8e1c0,0)
| | | | | | | | | | N_EXP_IDENT (5ef87d90,0)
| | | | | | | | | | | (ID="printf") TYPE:5ef87d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | | | N_EXP_STRING_LITERAL (5ef87cb0,0)
| | | | | | | | | | | | LITERAL: "%d"
| | | | | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | | | | N_EXP_EQL (5ef87ac0,0)
| | | | | | | | | | | | | N_EXP_IDENT (5ef87b50,1)
| | | | | | | | | | | | | | (ID="a") TYPE:5ef87b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | | | | | N_EXP_CAST (5ef87b50,0)
| | | | | | | | | | | | | | (float)
| | | | | | | | | | | | | | N_EXP_IDENT (5ef87be0,1)
| | | | | | | | | | | | | | | (ID="c") TYPE:5ef87be0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
| | | | | | | | | | | | N_ARG_LIST_NIL (0,0)
```

```

virtual-machine:~$ cat a.asm
    INT 0, 44
    SUP 0, main
    RET 0, 0
main:
    INT 0, 24
    LDA 1, 12
    LOD 0, 12
    STX 0, 0
    POP 0, 1
    LDA 1, 16
    LOD 0, 16
    STX 0, 0
    POP 0, 1
    LDA 1, 20
    LITI 0, 99
    STXB 0, 0
    POP 0, 1
    INT 0, 12
    LDA 0, 20
    LOD 1, 12
    LOD 1, 16
    EQLF 0, 0
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
    INT 0, 12
    LDA 0, 28
    LOD 1, 12
    LOD 1, 16
    NEQF 0, 0
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
    INT 0, 12
    LDA 0, 36
    LOD 1, 12
    LOD 1, 20
    CVTF 0, 0
    EQLF 0, 0
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
    RET 0, 0
.literal 12 0.000000
.literal 16 0.000000
.literal 20 "%d "
.literal 28 "%d "
.literal 36 "%d"

```

```
-virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,44
1: SUP 0,3
2: RET 0,0
3: INT 0,24
4: LDA 1,12
5: LOD 0,12
6: STX 0,0
7: POP 0,1
8: LDA 1,16
9: LOD 0,16
10: STX 0,0
11: POP 0,1
12: LDA 1,20
13: LITI 0,99
14: STXB 0,0
15: POP 0,1
16: INT 0,12
17: LDA 0,20
18: LOD 1,12
19: LOD 1,16
20: EQLF 0,0
21: POP 0,5
22: ADDR 0,-1
23: CAL 0,0
24: INT 0,12
25: LDA 0,28
26: LOD 1,12
27: LOD 1,16
28: NEQF 0,0
29: POP 0,5
30: ADDR 0,-1
31: CAL 0,0
32: INT 0,12
33: LDA 0,36
34: LOD 1,12
35: LOD 1,20
36: CVTF 0,0
37: EQLF 0,0
38: POP 0,5
39: ADDR 0,-1
40: CAL 0,0
41: RET 0,0
start execution
1065353216 0 0end execution
```

[설명]

실수 리터럴 같은 경우는 값이 제대로 저장이 되어 있지 않아 결과가 부정확한 것을 확인할 수 있다. a와 c의 타입이 동일하지 않아 cast 연산이 일어난 것을 시멘틱 트리를 통해 확인된다. 어셈블리 코드는 제대로 된 결과가 출력된다. float 타입을 고려한 연산을 구분하고, 1byte 연산을 구분해서 제대로 출력 결과를 보여준다.

16.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(){
    int a,b;
    a=2; b=2;
    printf("%d ",a==b);
    printf("%d ",a!=b);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
-virtual-machine:~$ cat a.asm
                                INT  0, 28
                                SUP  0, main
                                RET  0, 0
main:
                                INT  0, 20
                                LDA  1, 12
                                LITI 0, 2
                                STX  0, 0
                                POP  0, 1
                                LDA  1, 16
                                LITI 0, 2
                                STX  0, 0
                                POP  0, 1
                                INT  0, 12
                                LDA  0, 12
                                LOD  1, 12
                                LOD  1, 16
                                EQLI 0, 0
                                POP  0, 5
                                ADDR 0, printf
                                CAL  0, 0
                                INT  0, 12
                                LDA  0, 20
                                LOD  1, 12
                                LOD  1, 16
                                NEQI 0, 0
                                POP  0, 5
                                ADDR 0, printf
                                CAL  0, 0
                                RET  0, 0
.literal 12 "%d "
.literal 20 "%d "
```

```

virtual-machine:~$ ./interpreter a.as
===== symbol =====
  1: printf  -1
  2: malloc  -2
  3: scanf   -3
  4: main     3
===== code =====
  0: INT      0,28
  1: SUP      0,3
  2: RET      0,0
  3: INT      0,20
  4: LDA      1,12
  5: LITI     0,2
  6: STX      0,0
  7: POP      0,1
  8: LDA      1,16
  9: LITI     0,2
10: STX      0,0
11: POP      0,1
12: INT      0,12
13: LDA      0,12
14: LOD      1,12
15: LOD      1,16
16: EQLI     0,0
17: POP      0,5
18: ADDR     0,-1
19: CAL      0,0
20: INT      0,12
21: LDA      0,20
22: LOD      1,12
23: LOD      1,16
24: NEQI     0,0
25: POP      0,5
26: ADDR     0,-1
27: CAL      0,0
28: RET      0,0
start execution
1 0 end execution

```

[설명]

시멘틱 트리 구조는 cast 연산을 제외하면 이전 결과와 유사하여 생략하였다. 두 개의 피연산자 모두 int 타입이기 때문에 연산(EQLI, NEQI)은 모두 int 타입이다. interpreter의 결과에서 연산의 결과가 잘 출력되는 것을 확인할 수 있다.

17.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(){
    int a,b;
    a=0; b=12;
    printf("%d\n",a&&b);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
| N_STMT_LIST (0,0)
| | N_STMT_EXPRESSION (0,0)
| | | N_EXP_FUNCTION_CALL (4b46ac70,0)
| | | | N_EXP_AMP (4b4708b0,0)
| | | | | N_EXP_IDENT (4b46ad90,0)
| | | | | (ID="printf") TYPE:4b46ad90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | N_ARG_LIST (0,8)
| | | | | N_EXP_STRING_LITERAL (4b46acb0,0)
| | | | | | LITERAL: "%d\n"
| | | | | N_ARG_LIST (0,4)
| | | | | | N_EXP_AND (4b46aac0,0)
| | | | | | | N_EXP_IDENT (4b46aac0,1)
| | | | | | | (ID="a") TYPE:4b46aac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | N_EXP_IDENT (4b46aac0,1)
| | | | | | | (ID="b") TYPE:4b46aac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | | | N_ARG_LIST_NIL (0,0)
| | N_STMT_LIST_NIL (0,0)
```

```
-virtual-machine:~$ cat a.asm
        INT    0, 20
        SUP    0, main
        RET    0, 0

main:

        INT    0, 20
        LDA    1, 12
        LITI   0, 0
        STX    0, 0
        POP    0, 1
        LDA    1, 16
        LITI   0, 12
        STX    0, 0
        POP    0, 1
        INT    0, 12
        LDA    0, 12
        LOD    1, 12
        JPCR   0, L1
        LOD    1, 16

L1:

        POP    0, 5
        ADDR   0, printf
        CAL    0, 0
        RET    0, 0

.literal 12 "%d\n"
```



```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
 1: printf  -1
 2: malloc  -2
 3: scanf   -3
 4: main     3
 5: L1       17
===== code =====
 0: INT      0,20
 1: SUP      0,3
 2: RET      0,0
 3: INT      0,20
 4: LDA      1,12
 5: LITI     0,0
 6: STX      0,0
 7: POP      0,1
 8: LDA      1,16
 9: LITI     0,12
10: STX      0,0
11: POP      0,1
12: INT      0,12
13: LDA      0,12
14: LOD      1,12
15: JPCR     0,17
16: LOD      1,16
17: POP      0,5
18: ADDR     0,-1
19: CAL      0,0
20: RET      0,0
start execution
0
end execution

```

[설명]

시멘틱 트리를 통해 AND 연산에 필요한 왼쪽 수식과 오른쪽 수식이 모두 identifier인 것을 확인할 수 있다. AND 연산에서는 identifier의 값을 가져오고(LOD), 왼쪽 수식에서 거짓이 발생하면 JPCR를 하게 된다. 아닐 시에는 오른쪽 수식도 참인지를 검사한다. 이와 관련된 코드가 적절하게 출력되는 것을 확인할 수 있으며, 수식의 결과 역시도 적절한 결과를 출력하고 있다.

18.

[입력값]

```

virtual-machine:~$ cat ex.c
int main(){
    int a,b;
    a=0; b=1;
    printf("%d\n",a||b);
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
| N_EXP_FUNCTION_CALL (73e69c70,0)
| | N_EXP_AMP (73e6f8b0,0)
| | | N_EXP_IDENT (73e69d90,0)
| | | | (ID="printf") TYPE:73e69d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | N_ARG_LIST (0,8)
| | | N_EXP_STRING_LITERAL (73e69cb0,0)
| | | | LITERAL: "%d\n"
| | | N_ARG_LIST (0,4)
| | | | N_EXP_AND (73e69ac0,0)
| | | | | N_EXP_IDENT (73e69ac0,1)
| | | | | | (ID="a") TYPE:73e69ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | N_EXP_IDENT (73e69ac0,1)
| | | | | | (ID="b") TYPE:73e69ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | N_ARG_LIST_NIL (0,0)
N_STMT_LIST_NIL (0,0)
```

```
virtual-machine:~$ cat a.asm

        INT 0, 20
        SUP 0, main
        RET 0, 0
main:
        INT 0, 20
        LDA 1, 12
        LITI 0, 0
        STX 0, 0
        POP 0, 1
        LDA 1, 16
        LITI 0, 1
        STX 0, 0
        POP 0, 1
        INT 0, 12
        LDA 0, 12
        LOD 1, 12
        JPTR 0, L1
        LOD 1, 16
L1:
        POP 0, 5
        ADDR 0, printf
        CAL 0, 0
        RET 0, 0
.literal 12 "%d\n"
```

```

-vmachine:~$ ./interpreter a.asm
===== symbol =====
 1: printf  -1
 2: malloc  -2
 3: scanf   -3
 4: main     3
 5: L1       17
===== code =====
 0: INT      0,20
 1: SUP      0,3
 2: RET      0,0
 3: INT      0,20
 4: LDA      1,12
 5: LITI     0,0
 6: STX      0,0
 7: POP      0,1
 8: LDA      1,16
 9: LITI     0,1
10: STX      0,0
11: POP      0,1
12: INT      0,12
13: LDA      0,12
14: LOD      1,12
15: JPTR     0,17
16: LOD      1,16
17: POP      0,5
18: ADDR     0,-1
19: CAL      0,0
20: RET      0,0
start execution
1
end execution

```

[설명]

이전의 결과와 유사하게 OR 연산은 왼쪽 수식, 오른쪽 수식 모두 identifier이다. 각 identifier의 값을 LOD 해서 참과 거짓을 구분 짓는데, 왼쪽 수식부터 오른쪽 수식 순으로 연산이 된다. 만약 왼쪽 수식에서 참이 발생하면 참이 되고 JPCR을 통해 연산 다음 위치로 점프한다. 해당 결과가 참이므로 1이 출력되고 있다.

19.

[입력값]

```

-vmachine:~$ cat ex.c
void hamsoo(char arr[]){
    char c = arr[2];
}

int main(){
    char ch[20];
    hamsoo(ch);
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_STMT_COMPOUND (0,20)
| (ID="ch") TYPE:751bc610 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | TYPE
| | | ARRAY
| | | | INDEX
| | | | | INT=20
| | | | | ELEMENT_TYPE
| | | | | (char 1)
| N_STMT_LIST (0,0)
| | N_STMT_EXPRESSION (0,0)
| | | N_EXP_FUNCTION_CALL (751b6c70,0)
| | | | N_EXP_AMP (751bc8d0,0)
| | | | | N_EXP_IDENT (751bc220,0)
| | | | | | (ID="hamsoo") TYPE:751bc220 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | N_ARG_LIST (0,4)
| | | | | | N_EXP_CAST (751bc950,0)
| | | | | | | (DONE:751bc950)
| | | | | | | N_EXP_IDENT (751bc610,0)
| | | | | | | | (ID="ch") TYPE:751bc610 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | N_ARG_LIST_NIL (0,0)
| | N_STMT_LIST_NIL (0,0)
```

```
-virtual-machine:~$ cat a.asm

        INT    0, 12
        SUP    0, main
        RET    0, 0

hamsoo:

        INT    0, 20
        RET    0, 0

main:

        INT    0, 32
        INT    0, 12
        LDA    1, 12
        POP    0, 4
        ADDR   0, hamsoo
        CAL    0, 0
        RET    0, 0
```

```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 5
5: hamsoo 3
===== code =====
0: INT 0,12
1: SUP 0,5
2: RET 0,0
3: INT 0,20
4: RET 0,0
5: INT 0,32
6: INT 0,12
7: LDA 1,12
8: POP 0,4
9: ADDR 0,3
10: CAL 0,0
11: RET 0,0
start execution
end execution

```

[설명]

hamsoo 집입시에 INT의 값은 파라미터에 존재하는 배열 포인터(4)와 지역변수 char ch[20], char c(aligned 4)로 12+4+20+4인 최종 40의 활성화 레코드 공간이 주어진다. 그리고 파라미터는 해당 배열의 시작 주소(배열의 주소)를 전달해준다. 또한, char c에 관한 내용은 사용되지 않아 출력되지 않는 것을 확인할 수 있다. 적절한 결과가 출력된다.

20.

[입력값]

```

virtual-machine:~$ cat ex.c
int main(){
    float fl;
    fl=3.3;
    if(fl){ printf("%f", fl); }
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_STMT_COMPOUND (0,4)
| (ID="fl") TYPE:16441b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | TYPE
| | | (float)
| | N_STMT_LIST (0,0)
| | | N_STMT_EXPRESSION (0,0)
| | | | N_EXP_ASSIGN (16441b50,0)
| | | | | N_EXP_IDENT (16441b50,1)
| | | | | (ID="fl") TYPE:16441b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | N_EXP_FLOAT_CONST (16441b50,0)
| | | | | LITERAL: 272008302207532160516096.000000
| | | N_STMT_LIST (0,0)
| | | | N_STMT_IF (0,0)
| | | | | N_EXP_CAST (16441ac0,0)
| | | | | (int)
| | | | | N_EXP_IDENT (16441b50,1)
| | | | | (ID="fl") TYPE:16441b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | N_STMT_COMPOUND (0,0)
| | | | | N_STMT_LIST (0,0)
| | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | N_EXP_FUNCTION_CALL (16441c70,0)
| | | | | | | | N_EXP_AMP (16447840,0)
| | | | | | | | | N_EXP_IDENT (16441d90,0)
| | | | | | | | | (ID="printf") TYPE:16441d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | N_ARG_LIST (0,8)
| | | | | | | | | N_EXP_STRING_LITERAL (16441cb0,0)
| | | | | | | | | | LITERAL: "%f"
| | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | N_EXP_IDENT (16441b50,1)
| | | | | | | | | (ID="fl") TYPE:16441b50 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | N_STMT_LIST_NIL (0,0)
| | N_STMT_LIST_NIL (0,0)
```

```
-virtual-machine:~$ cat a.asm

        INT    0, 24
        SUP    0, main
        RET    0, 0

main:

        INT    0, 16
        LDA    1, 12
        LOD    0, 12
        STX    0, 0
        POP    0, 1
        LOD    1, 12
        CVTI    0, 0
        JPC    0, L1
        INT    0, 12
        LDA    0, 16
        LOD    1, 12
        POP    0, 5
        ADDR    0, printf
        CAL    0, 0

L1:

        RET    0, 0

.literal 12 272008302207532160516096.000000
.literal 16 "%f"
```



```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
  1: printf  -1
  2: malloc  -2
  3: scanf   -3
  4: main     3
  5: L1       17
===== code =====
  0: INT      0,24
  1: SUP      0,3
  2: RET      0,0
  3: INT      0,16
  4: LDA      1,12
  5: LOD      0,12
  6: STX      0,0
  7: POP      0,1
  8: LOD      1,12
  9: CVTI     0,0
10: JPC      0,17
11: INT      0,12
12: LDA      0,16
13: LOD      1,12
14: POP      0,5
15: ADDR     0,-1
16: CAL      0,0
17: RET      0,0
start execution
-2.000000end execution

```

[설명]

if문에서 조건 수식은 int 타입이어야 하기 때문에, float 타입의 변수가 조건식에서 판단이 되기 전에 CVTI가 된다. 이후 조건이 거짓일 시에는 JPC로 if문 이후(L1)로 점프하게 된다.

21.

[입력값]

```

virtual-machine:~$ cat ex.c
int main(){
    char ch;
    ch='\0';
    if(ch){ printf("%c\n", ch); }
    else { printf("%s\n","----else----"); }
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_STMT_LIST (0,0)
| N_STMT_IF_ELSE (0,0)
| | N_EXP_IDENT (9e0dbe0,1)
| | | (ID="ch") TYPE:9e0dbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | N_STMT_COMPOUND (0,0)
| | | N_STMT_LIST (0,0)
| | | | N_STMT_EXPRESSION (0,0)
| | | | | N_EXP_FUNCTION_CALL (9e0dc70,0)
| | | | | N_EXP_AMP (9e13b00,0)
| | | | | N_EXP_IDENT (9e0dd90,0)
| | | | | | (ID="printf") TYPE:9e0dd90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | N_ARG_LIST (0,8)
| | | | | | N_EXP_STRING_LITERAL (9e0dcb0,0)
| | | | | | | LITERAL: "%c\n"
| | | | | | N_ARG_LIST (0,4)
| | | | | | | N_EXP_IDENT (9e0dbe0,1)
| | | | | | | | (ID="ch") TYPE:9e0dbe0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | N_ARG_LIST_NIL (0,0)
| | | | N_STMT_LIST_NIL (0,0)
| | N_STMT_COMPOUND (0,0)
| | | N_STMT_LIST (0,0)
| | | | N_STMT_EXPRESSION (0,0)
| | | | | N_EXP_FUNCTION_CALL (9e0dc70,0)
| | | | | N_EXP_AMP (9e13b80,0)
| | | | | N_EXP_IDENT (9e0dd90,0)
| | | | | | (ID="printf") TYPE:9e0dd90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | N_ARG_LIST (0,8)
| | | | | | N_EXP_STRING_LITERAL (9e0dcb0,0)
| | | | | | | LITERAL: "%s\n"
| | | | | | N_ARG_LIST (0,4)
| | | | | | | N_EXP_STRING_LITERAL (9e0dcb0,0)
| | | | | | | | LITERAL: "----else----"
| | | | | | | N_ARG_LIST_NIL (0,0)
| | | | N_STMT_LIST_NIL (0,0)
| N_STMT_LIST_NIL (0,0)
```

```
-virtual-machine:~$ cat a.asm

    INT 0, 44
    SUP 0, main
    RET 0, 0
main:
    INT 0, 16
    LDA 1, 12
    LITI 0, 0
    STXB 0, 0
    POP 0, 1
    LOD 1, 12
    JPC 0, L1
    INT 0, 12
    LDA 0, 12
    LOD 1, 12
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
    JMP 0, L2
L1:
    INT 0, 12
    LDA 0, 20
    LDA 0, 28
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
L2:
    RET 0, 0
.literal 12 "%c\n"
.literal 20 "%s\n"
.literal 28 "----else----
```

```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
 1: printf  -1
 2: malloc  -2
 3: scanf   -3
 4: main     3
 5: L1      17
 6: L2      23
===== code =====
 0: INT      0,44
 1: SUP      0,3
 2: RET      0,0
 3: INT      0,16
 4: LDA      1,12
 5: LITI     0,0
 6: STXB     0,0
 7: POP      0,1
 8: LOD      1,12
 9: JPC      0,17
10: INT      0,12
11: LDA      0,12
12: LOD      1,12
13: POP      0,5
14: ADDR     0,-1
15: CAL      0,0
16: JMP      0,23
17: INT      0,12
18: LDA      0,20
19: LDA      0,28
20: POP      0,5
21: ADDR     0,-1
22: CAL      0,0
23: RET      0,0
start execution
----else----
end execution

```

[설명]

if-else 문에서는 조건 수식이 참인 경우 해당 compound_statement를 수행하고 else 문을 건너뛴 L2로 점프하게 된다. 만약 수식이 거짓인 경우 else문(L1)으로 점프해서 해당 내용을 수행하게 된다. 작성한 코드에서는 조건 수식이 거짓이므로 else문이 수행되는 것을 확인할 수 있다. ./compiler와 ./interpreter 둘 다 적절한 결과를 출력하고 있다.

22.

[입력값]

```

virtual-machine:~$ cat ex.c
int main(){
    int i;
    i=3;
    while(i--){
        printf("%d\n",i);
    }
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
| N_STMT_WHILE (0,0)
| | N_EXP_POST_DEC (5e420ac0,0)
| | | N_EXP_IDENT (5e420ac0,1)
| | | | (ID="i") TYPE:5e420ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | N_STMT_COMPOUND (0,0)
| | | N_STMT_LIST (0,0)
| | | | N_STMT_EXPRESSION (0,0)
| | | | | N_EXP_FUNCTION_CALL (5e420c70,0)
| | | | | | N_EXP_AMP (5e426800,0)
| | | | | | | N_EXP_IDENT (5e420d90,0)
| | | | | | | | (ID="printf") TYPE:5e420d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | N_ARG_LIST (0,8)
| | | | | | | N_EXP_STRING_LITERAL (5e420cb0,0)
| | | | | | | | LITERAL: "%d\n"
| | | | | | N_ARG_LIST (0,4)
| | | | | | | N_EXP_IDENT (5e420ac0,1)
| | | | | | | | (ID="i") TYPE:5e420ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | N_ARG_LIST_NIL (0,0)
| | | N_STMT_LIST_NIL (0,0)
| N_STMT_LIST_NIL (0,0)
```

```
-virtual-machine:~$ cat a.asm
        INT    0, 20
        SUP    0, main
        RET    0, 0
main:
        INT    0, 16
        LDA    1, 12
        LITI   0, 3
        STX    0, 0
        POP    0, 1
L2:
        LOD    1, 12
        LDA    1, 12
        LDX    0, 1
        DECI   0, 0
        STO    0, 0
        JPC    0, L3
        INT    0, 12
        LDA    0, 12
        LOD    1, 12
        POP    0, 5
        ADDR   0, printf
        CAL    0, 0
L1:
        JMP    0, L2
L3:
        RET    0, 0
.literal 12  "%d\n"
```

```

~virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
5: L2 8
6: L3 21
7: L1 20
===== code =====
0: INT 0,20
1: SUP 0,3
2: RET 0,0
3: INT 0,16
4: LDA 1,12
5: LITI 0,3
6: STX 0,0
7: POP 0,1
8: LOD 1,12
9: LDA 1,12
10: LDX 0,1
11: DECI 0,0
12: STO 0,0
13: JPC 0,21
14: INT 0,12
15: LDA 0,12
16: LOD 1,12
17: POP 0,5
18: ADDR 0,-1
19: CAL 0,0
20: JMP 0,8
21: RET 0,0
start execution
2
1
0
end execution

```

[설명]

while문에서는 조건 수식이 참이면 compound_statement를 수행하고 while문의 조건 수식 부분(L2)로 점프하게 된다. 만약 조건 수식이 거짓이면 while문 종료 시점(L3)으로 점프하게 된다. 해당 조건 수식에서는 후위 감소가 발생하고 있다. i는 lvalue로 자기 자신의 주소를 불러와서 자신의 값을 감소시킨 값을 저장하게 된다. 매 조건마다 i를 감소하고 0이 되는 순간 while문 종료 시점으로 점프하는 것을 위의 그림을 통해 확인할 수 있다.

23.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(){
    int i;
    char *ch;
    i=2;
    ch="do-while";
    do{
        printf("%s\n", ch);
    }while(i--);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
graph TD
    N_STMT_LIST_00[N_STMT_LIST (0,0)] --> N_STMT_DO_00[N_STMT_DO (0,0)]
    N_STMT_DO_00 --> N_STMT_COMPOUND_00[N_STMT_COMPOUND (0,0)]
    N_STMT_COMPOUND_00 --> N_STMT_LIST_01[N_STMT_LIST (0,0)]
    N_STMT_LIST_01 --> N_STMT_EXPRESSION_00[N_STMT_EXPRESSION (0,0)]
    N_STMT_EXPRESSION_00 --> N_EXP_FUNCTION_CALL_00[N_EXP_FUNCTION_CALL (7de20c70,0)]
    N_EXP_FUNCTION_CALL_00 --> N_EXP_AMP_00[N_EXP_AMP (7de26a30,0)]
    N_EXP_AMP_00 --> N_EXP_IDENT_00[N_EXP_IDENT (7de20d90,0)]
    N_EXP_IDENT_00 --> ID_printf["(ID='printf') TYPE:7de20d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0"]
    ID_printf --> N_ARG_LIST_00[N_ARG_LIST (0,8)]
    N_ARG_LIST_00 --> N_EXP_STRING_LITERAL_00[N_EXP_STRING_LITERAL (7de20cb0,0)]
    N_EXP_STRING_LITERAL_00 --> LITERAL_printf["LITERAL: '%s\\n'"]
    LITERAL_printf --> N_ARG_LIST_01[N_ARG_LIST (0,4)]
    N_ARG_LIST_01 --> N_EXP_IDENT_01[N_EXP_IDENT (7de26260,1)]
    N_EXP_IDENT_01 --> ID_ch["(ID='ch') TYPE:7de26260 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16"]
    ID_ch --> N_ARG_LIST_NIL_00[N_ARG_LIST_NIL (0,0)]
    N_ARG_LIST_NIL_00 --> N_STMT_LIST_NIL_00[N_STMT_LIST_NIL (0,0)]
    N_STMT_LIST_NIL_00 --> N_EXP_POST_DEC_00[N_EXP_POST_DEC (7de20ac0,0)]
    N_EXP_POST_DEC_00 --> N_EXP_IDENT_02[N_EXP_IDENT (7de20ac0,1)]
    N_EXP_IDENT_02 --> ID_i["(ID='i') TYPE:7de20ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12"]
    ID_i --> N_STMT_LIST_NIL_01[N_STMT_LIST_NIL (0,0)]
```

```
-virtual-machine:~$ cat a.asm
        INT    0, 32
        SUP    0, main
        RET    0, 0

main:
        INT    0, 20
        LDA    1, 12
        LITI   0, 2
        STX    0, 0
        POP    0, 1
        LDA    1, 16
        LDA    0, 12
        STX    0, 0
        POP    0, 1

L3:
```

```

L3:
        INT 0, 12
        LDA 0, 24
        LOD 1, 16
        POP 0, 5
        ADDR 0, printf
        CAL 0, 0

L2:
        LOD 1, 12
        LDA 1, 12
        LDX 0, 1
        DECI 0, 0
        STO 0, 0
        JPT 0, L3

L1:
        RET 0, 0

.literal 12 "do-while"
.literal 24 "%s\n"

```

```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
5: L3 12
6: L2 18
7: L1 24
===== code =====
0: INT 0,32
1: SUP 0,3
2: RET 0,0
3: INT 0,20
4: LDA 1,12
5: LITI 0,2
6: STX 0,0
7: POP 0,1
8: LDA 1,16
9: LDA 0,12
10: STX 0,0
11: POP 0,1
12: INT 0,12
13: LDA 0,24
14: LOD 1,16
15: POP 0,5
16: ADDR 0,-1
17: CAL 0,0
18: LOD 1,12
19: LDA 1,12
20: LDX 0,1
21: DECI 0,0
22: STO 0,0
23: JPT 0,12
24: RET 0,0
start execution
do-while
do-while
do-while
end execution

```

[설명]

do-while문에서는 compound_statement에 있는 내용을 수행하고 그 다음에 조건 수식을 검사하게 된다. 해당 수식이 참이면 do의 시작 부분(L3) 부분으로 점프하게 된다. 거짓이면 다음 L1 부분으로 넘어가게 된다. 조건이 참일 동안 “do-while” 스트링 리터럴을 출력하게 하는데, 위의 그림에서 적절한 결과가 나오는 것을 확인할 수 있다.

24.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(){
    int i;
    i=3;
    for(i ; i>=0 ; --i){
        if(i==1) break;
        printf("%d\n", i);
    }
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
| N_STMT_FOR (0,0)
| | N_FOR_EXP (0,0)
| | | N_EXP_IDENT (4dbdcac0,1)
| | | | (ID="i") TYPE:4dbdcac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | N_EXP_GEQ (4dbdcac0,0)
| | | | N_EXP_IDENT (4dbdcac0,1)
| | | | | (ID="i") TYPE:4dbdcac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | N_EXP_INT_CONST (4dbdcac0,0)
| | | | | INT=0
| | | N_EXP_PRE_DEC (4dbdcac0,0)
| | | | N_EXP_IDENT (4dbdcac0,1)
| | | | | (ID="i") TYPE:4dbdcac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | N_STMT_COMPOUND (0,0)
| | | N_STMT_LIST (0,0)
| | | | N_STMT_IF (0,0)
| | | | | N_EXP_EQ (4dbdcac0,0)
| | | | | | N_EXP_IDENT (4dbdcac0,1)
| | | | | | | (ID="i") TYPE:4dbdcac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | N_EXP_INT_CONST (4dbdcac0,0)
| | | | | | | INT=1
| | | | | N_STMT_BREAK (0,0)
| | | | N_STMT_LIST (0,0)
| | | | | N_STMT_EXPRESSION (0,0)
| | | | | | N_EXP_FUNCTION_CALL (4dbdcc70,0)
| | | | | | | N_EXP_AMP (4dbe2ac0,0)
| | | | | | | N_EXP_IDENT (4dbdcd90,0)
| | | | | | | | (ID="printf") TYPE:4dbdcd90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | N_ARG_LIST (0,8)
| | | | | | | | N_EXP_STRING_LITERAL (4dbdccb0,0)
| | | | | | | | | LITERAL: "%d\n"
| | | | | | | | N_ARG_LIST (0,4)
| | | | | | | | | N_EXP_IDENT (4dbdcac0,1)
| | | | | | | | | | (ID="i") TYPE:4dbdcac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | N_STMT_LIST_NIL (0,0)
| N_STMT_LIST_NIL (0,0)
```

```
-virtual-machine:~$ cat a.asm

    INT 0, 20
    SUP 0, main
    RET 0, 0
main:
    INT 0, 16
    LDA 1, 12
    LITI 0, 3
    STX 0, 0
    POP 0, 1
    LOD 1, 12
    POP 0, 1
L2:
    LOD 1, 12
    LITI 0, 0
    GEQI 0, 0
    JPC 0, L3
    LOD 1, 12
    LITI 0, 1
    EQLI 0, 0
    JPC 0, L4
    JMP 0, L3
L4:
    INT 0, 12
    LDA 0, 12
    LOD 1, 12
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
L1:
    LDA 1, 12
    LDX 0, 1
    DECI 0, 0
    STX 0, 0
    POP 0, 1
    JMP 0, L2
L3:
    RET 0, 0
.literal 12 "%d\n"
```

```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
 1: printf  -1
 2: malloc  -2
 3: scanf   -3
 4: main     3
 5: L2      10
 6: L3      31
 7: L4      19
 8: L1      25
===== code =====
 0: INT      0,20
 1: SUP      0,3
 2: RET      0,0
 3: INT      0,16
 4: LDA      1,12
 5: LITI     0,3
 6: STX      0,0
 7: POP      0,1
 8: LOD      1,12
 9: POP      0,1
10: LOD      1,12
11: LITI     0,0
12: GEQI     0,0
13: JPC      0,31
14: LOD      1,12
15: LITI     0,1
16: EQLI     0,0
17: JPC      0,19
18: JMP      0,31
19: INT      0,12
20: LDA      0,12
21: LOD      1,12
22: POP      0,5
23: ADDR     0,-1
24: CAL      0,0
25: LDA      1,12
26: LDX      0,1
27: DECI     0,0
28: STX      0,0
29: POP      0,1
30: JMP      0,10
31: RET      0,0
start execution
3
2
end execution

```

[설명]

for문은 초기값을 가져오고 조건을 확인한 후, 조건이 참일 경우 compound_statement를 수행하고 증감식을 수행한다. 이후 다시 조건을 확인하는 단계로 이동해서 조건이 거짓일 때까지 반복한다. 어셈블리어를 보면, L2에서는 조건 수식이 참인지를 확인하고 조건이 거짓이면 for문 종료된 지점 L3로 이동한다. 그리고 if문에서 i가 1인지에 대한 조건을 확인하는데, 이때 거짓이면 L4로 점프해서 compound_statement를 이어가고, 참인 경우 L3로 점프해서 종료하게 된다. L1은 i의 전위 감소 과정이다. 결과가 적절하게 출력된 것을 확인할 수 있다.

25.

[입력값]

```
virtual-machine:~$ cat ex.c
int main(){
    int i;
    i=3;
    while(i--){
        if(i) continue;
        printf("%d\n", i);
    }
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_STMT_WHILE (0,0)
| N_EXP_POST_DEC (b04c0ac0,0)
| | N_EXP_IDENT (b04c0ac0,1)
| | | (ID="i") TYPE:b04c0ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| N_STMT_COMPOUND (0,0)
| | N_STMT_LIST (0,0)
| | | N_STMT_IF (0,0)
| | | | N_EXP_IDENT (b04c0ac0,1)
| | | | | (ID="i") TYPE:b04c0ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | N_STMT_CONTINUE (0,0)
| | | N_STMT_LIST (0,0)
| | | | N_STMT_EXPRESSION (0,0)
| | | | | N_EXP_FUNCTION_CALL (b04c0c70,0)
| | | | | | N_EXP_AMP (b04c6900,0)
| | | | | | | N_EXP_IDENT (b04c0d90,0)
| | | | | | | | (ID="printf") TYPE:b04c0d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | N_ARG_LIST (0,8)
| | | | | | | | N_EXP_STRING_LITERAL (b04c0cb0,0)
| | | | | | | | | LITERAL: "%d\n"
| | | | | | | N_ARG_LIST (0,4)
| | | | | | | | N_EXP_IDENT (b04c0ac0,1)
| | | | | | | | | (ID="i") TYPE:b04c0ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | | N_ARG_LIST_NIL (0,0)
| | | N_STMT_LIST_NIL (0,0)
N_STMT_LIST_NIL (0,0)
```



```
-virtual-machine:~$ cat a.asm
```

```

    INT 0, 20
    SUP 0, main
    RET 0, 0
main:
    INT 0, 16
    LDA 1, 12
    LITI 0, 3
    STX 0, 0
    POP 0, 1
L2:
    LOD 1, 12
    LDA 1, 12
    LDX 0, 1
    DECI 0, 0
    STO 0, 0
    JPC 0, L3
    LOD 1, 12
    JPC 0, L4
    JMP 0, L1
L4:
    INT 0, 12
    LDA 0, 12
    LOD 1, 12
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
L1:
    JMP 0, L2
L3:
    RET 0, 0
.literal 12 "%d\n"
```

```

virtual-machine:~$ ./interpreter a.
===== symbol =====
  1: printf    -1
  2: malloc    -2
  3: scanf     -3
  4: main       3
  5: L2         8
  6: L3        24
  7: L4        17
  8: L1        23
===== code =====
  0: INT      0,20
  1: SUP      0,3
  2: RET      0,0
  3: INT      0,16
  4: LDA      1,12
  5: LITI     0,3
  6: STX      0,0
  7: POP      0,1
  8: LOD      1,12
  9: LDA      1,12
10: LDX      0,1
11: DECI     0,0
12: STO      0,0
13: JPC      0,24
14: LOD      1,12
15: JPC      0,17
16: JMP      0,23
17: INT      0,12
18: LDA      0,12
19: LOD      1,12
20: POP      0,5
21: ADDR     0,-1
22: CAL      0,0
23: JMP      0,8
24: RET      0,0
start execution
0
end execution

```

[설명]

L2는 조건 수식(후위 감소식)을 수행하고 조건을 확인한다. 조건이 거짓이면 while이 종료된 시점(L3)로 점프하고, 조건이 참이면 if문을 통해 I가 참인지를 확인한다. 거짓이면 compound_statement를 이어서 수행하고 참이면 while문의 마지막 부분(L1)으로 점프해서 다시 while문의 처음 부분(L2)로 점프하게 된다. 위의 그림을 통해 적절한 출력 결과를 보여주고 있음을 알 수 있다.

26.

[입력값]

```
-virtual-machine:~$ cat ex.c
int i;
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
===== semantic tree =====
N_PROGRAM (0,16)
| (ID="i") TYPE:f5253ac0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=12
| | TYPE
| | | (int)
start code generation
end code generation
```

```
-virtual-machine:~$ cat a.asm

        INT    0, 16
        SUP    0, main
        RET    0, 0
```

```
-virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf  -3
4: main   0
error: undefined identifier main
```

[설명]

전역변수 I로 인해서 12+4의 크기만큼 INT되고 main으로 점프하려고 한다. 하지만 main 함수가 존재하지 않아 error message가 출력된다.

27.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(int a, float b, char c, char *d){
    char e[15];
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
===== semantic tree =====
N_PROGRAM (0,12)
| (ID="main") TYPE:d5d2a3d0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:d5d24ac0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=12
| | | | | | TYPE
| | | | | | | (int)
| | | | | (ID="b") TYPE:d5d24b50 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=16
| | | | | | TYPE
| | | | | | | (float)
| | | | | (ID="c") TYPE:d5d24ac0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=20
| | | | | | TYPE
| | | | | | | (int)
| | | | | (ID="d") TYPE:d5d2a340 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=24
| | | | | | TYPE
| | | | | | | POINTER
| | | | | | | | ELEMENT_TYPE
| | | | | | | | | (char 1)
| | | | | TYPE
| | | | | | (int)
| | | | | BODY
| | | | | | N_STMT_COMPOUND (0,16)
| | | | | | | (ID="e") TYPE:d5d2a4e0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=28
| | | | | | | | TYPE
| | | | | | | | | ARRAY
| | | | | | | | | | INDEX
| | | | | | | | | | | INT=15
| | | | | | | | | | | ELEMENT_TYPE
| | | | | | | | | | | | (char 1)
| | | | | | | | | N_STMT_LIST_NIL (0,0)
start code generation
end code generation
```

```
virtual-machine:~$ cat a.asm

        INT    0, 12
        SUP    0, main
        RET    0, 0

main:

        INT    0, 44
        RET    0, 0
```

```
virtual-machine:~$ ./interpreter a.asm

===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,12
1: SUP 0,3
2: RET 0,0
3: INT 0,44
4: RET 0,0
start execution
end execution
```

[설명]

전역변수나 리터럴 테이블에 저장된 내용이 없기 때문에 맨 처음에 나오는 INT는 12이다. 그리고 main 함수의 파라미터의 크기(16)와 내부의 지역 변수 크기(15의 align된 크기 -> 16)의 합 32에 12를 더해 44의 크기만큼 INT된다.

28.

[입력값]

```
-virtual-machine:~$ cat ex.c
int main(int a, float b, char c, char *d){
    static char e[15];
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
-virtual-machine:~$ cat a.asm
                INT  0, 28
                SUP  0, main
                RET  0, 0

main:
                INT  0, 28
                RET  0, 0

-virtual-machine:~$ ./interpreter a.asm
===== symbol =====
  1: printf    -1
  2: malloc    -2
  3: scanf     -3
  4: main      3
===== code =====
  0: INT       0,28
  1: SUP       0,3
  2: RET       0,0
  3: INT       0,28
  4: RET       0,0
start execution
end execution
```

[설명]

static으로 선언된 변수는 level 0(전역)으로 처리된다. 따라서 맨처음 INT는 12+16(15의 align된 값)이고 main 함수의 INT는 '파라미터의 크기+12'가 된다.

29.

[입력]

```
-virtual-machine:~$ cat ex.c
int main(){
    int a,b,c;
    a=b=c=1;
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_STMT_LIST (0,0)
| N_STMT_EXPRESSION (0,0)
| | N_EXP_ASSIGN (90536ac0,0)
| | | N_EXP_IDENT (90536ac0,1)
| | | | (ID="a") TYPE:90536ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | N_EXP_ASSIGN (90536ac0,0)
| | | | N_EXP_IDENT (90536ac0,1)
| | | | | (ID="b") TYPE:90536ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=16
| | | | N_EXP_ASSIGN (90536ac0,0)
| | | | | N_EXP_IDENT (90536ac0,1)
| | | | | | (ID="c") TYPE:90536ac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=20
| | | | | N_EXP_INT_CONST (90536ac0,0)
| | | | | | INT=1
| N_STMT_LIST_NIL (0,0)
```

```
virtual-machine:~$ cat a.asm
        INT 0, 12
        SUP 0, main
        RET 0, 0
main:
        INT 0, 24
        LDA 1, 12
        LDA 1, 16
        LDA 1, 20
        LITI 0, 1
        STX 0, 0
        STX 0, 0
        STX 0, 0
        POP 0, 1
        RET 0, 0
virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
===== code =====
0: INT 0,12
1: SUP 0,3
2: RET 0,0
3: INT 0,24
4: LDA 1,12
5: LDA 1,16
6: LDA 1,20
7: LITI 0,1
8: STX 0,0
9: STX 0,0
10: STX 0,0
11: POP 0,1
12: RET 0,0
start execution
end execution
```

[설명]

연속적인 치환 연산(ASSIGN)은 연속적인 변수들의 주소를 왼쪽에서 오른쪽 순으로 한번에 stack의 top으로 가져오고 값을 역순으로 저장하는 것을 확인할 수 있다. 따라서 적절한 결과를 출력하는 것을 알 수 있다.

30.

[입력]

```
-virtual-machine:~$ cat ex.c
int main(){
    int a=0;
    while(1){
        while(2){
            if(!a) break;
        }
        if(!a) break;
    }
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_STMT_WHILE (0,0)
| N_EXP_INT_CONST (2508cac0,0)
| | INT=1
| N_STMT_COMPOUND (0,0)
| | N_STMT_LIST (0,0)
| | | N_STMT_WHILE (0,0)
| | | | N_EXP_INT_CONST (2508cac0,0)
| | | | | INT=2
| | | | N_STMT_COMPOUND (0,0)
| | | | | N_STMT_LIST (0,0)
| | | | | N_STMT_IF (0,0)
| | | | | | N_EXP_NOT (2508cac0,0)
| | | | | | | N_EXP_IDENT (2508cac0,1)
| | | | | | | | (ID="a") TYPE:2508cac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | | | N_STMT_BREAK (0,0)
| | | | | | N_STMT_LIST_NIL (0,0)
| | | N_STMT_LIST (0,0)
| | | | N_STMT_IF (0,0)
| | | | | N_EXP_NOT (2508cac0,0)
| | | | | N_EXP_IDENT (2508cac0,1)
| | | | | | (ID="a") TYPE:2508cac0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=12
| | | | | N_STMT_BREAK (0,0)
| | | | N_STMT_LIST_NIL (0,0)
N_STMT_LIST_NIL (0,0)
```

```

virtual-machine:~$ cat a.asm
main:
    INT 0, 12
    SUP 0, main
    RET 0, 0
L2:
    LITI 0, 1
    JPC 0, L3
L5:
    LITI 0, 2
    JPC 0, L6
    LOD 1, 12
    NOT 0, 0
    JPC 0, L7
    JMP 0, L6
L7:
L4:
    JMP 0, L5
L6:
    LOD 1, 12
    NOT 0, 0
    JPC 0, L8
    JMP 0, L3
L8:
L1:
    JMP 0, L2
L3:
    RET 0, 0

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
5: L2 4
6: L3 18
7: L5 6
8: L6 13
9: L7 12
10: L4 12
11: L8 17
12: L1 17
===== code =====
0: INT 0,12
1: SUP 0,3
2: RET 0,0
3: INT 0,16
4: LITI 0,1
5: JPC 0,18
6: LITI 0,2
7: JPC 0,13
8: LOD 1,12
9: NOT 0,0
10: JPC 0,12
11: JMP 0,13
12: JMP 0,6
13: LOD 1,12
14: NOT 0,0
15: JPC 0,17
16: JMP 0,18
17: JMP 0,4
18: RET 0,0
start execution
end execution

```

[설명]

while문의 조건 수식에 들어가는 정수 값은 LITI를 통해 값을 불러오고 참 거짓을 구분해 점프를 하게 된다. 위의 이미지와 같이 적절하게 시멘틱 트리와 어셈블리 코드, interpreter 결과물이 출력되는 것을 확인할 수 있다.

31.

[입력]

```

virtual-machine:~$ cat ex.c
int factorial(int n) {
    printf("%d\n", n);
    if (n <= 1) return 1;
    return n * factorial(n-1);
}

int main() {
    int num = 2;
    return 0;
}

```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
N_STMT_COMPOUND (0,0)
| N_STMT_LIST (0,0)
| | N_STMT_EXPRESSION (0,0)
| | | N_EXP_FUNCTION_CALL (c8206c70,0)
| | | | N_EXP_AMP (c820cce0,0)
| | | | | N_EXP_IDENT (c8206d90,0)
| | | | | | (ID="printf") TYPE:c8206d90 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | N_ARG_LIST (0,8)
| | | | | N_EXP_STRING_LITERAL (c8206cb0,0)
| | | | | | LITERAL: "%d\n"
| | | | | N_ARG_LIST (0,4)
| | | | | | N_EXP_IDENT (c8206ac0,1)
| | | | | | | (ID="n") TYPE:c8206ac0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=12
| | | | | N_ARG_LIST_NIL (0,0)
| | N_STMT_LIST (0,0)
| | | N_STMT_IF (0,0)
| | | | N_EXP_LEQ (c8206ac0,0)
| | | | | N_EXP_IDENT (c8206ac0,1)
| | | | | | (ID="n") TYPE:c8206ac0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=12
| | | | | N_EXP_INT_CONST (c8206ac0,0)
| | | | | | INT=1
| | | | N_STMT_RETURN (0,0)
| | | | | N_EXP_INT_CONST (c8206ac0,0)
| | | | | | INT=1
| | | N_STMT_LIST (0,0)
| | | N_STMT_RETURN (0,0)
| | | | N_EXP_MUL (c8206ac0,0)
| | | | | N_EXP_IDENT (c8206ac0,1)
| | | | | | (ID="n") TYPE:c8206ac0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=12
| | | | | N_EXP_FUNCTION_CALL (c8206ac0,0)
| | | | | | N_EXP_AMP (c820cd60,0)
| | | | | | | N_EXP_IDENT (c820c1e0,0)
| | | | | | | | (ID="factorial") TYPE:c820c1e0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | N_ARG_LIST (0,4)
| | | | | | | N_EXP_SUB (c8206ac0,0)
| | | | | | | | N_EXP_IDENT (c8206ac0,1)
| | | | | | | | | (ID="n") TYPE:c8206ac0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=12
| | | | | | | N_EXP_INT_CONST (c8206ac0,0)
| | | | | | | | INT=1
| | | | | | N_ARG_LIST_NIL (0,0)
| | | N_STMT_LIST_NIL (0,0)
```

```

virtual-machine:~$ cat a.asm

        INT 0, 20
        SUP 0, main
        RET 0, 0
factorial:
        INT 0, 16
        INT 0, 12
        LDA 0, 12
        LOD 1, 12
        POP 0, 5
        ADDR 0, printf
        CAL 0, 0
        LOD 1, 12
        LITI 0, 1
        LEQI 0, 0
        JPC 0, L1
        LDA 1, -4
        LITI 0, 1
        STO 0, 0
        RET 0, 0
L1:
        LDA 1, -4
        LOD 1, 12
        INT 0, 16
        LOD 1, 12
        LITI 0, 1
        SUBI 0, 0
        POP 0, 4
        ADDR 0, factorial
        CAL 0, 0
        MULI 0, 0
        STO 0, 0
        RET 0, 0
        RET 0, 0
main:
        INT 0, 16
        LDA 1, -4
        LITI 0, 0
        STO 0, 0
        RET 0, 0
        RET 0, 0
.literal 12 "%d\n"

```

```

virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 31
5: factorial 3
6: L1 18
===== code =====
0: INT 0,20
1: SUP 0,31
2: RET 0,0
3: INT 0,16
4: INT 0,12
5: LDA 0,12
6: LOD 1,12
7: POP 0,5
8: ADDR 0,-1
9: CAL 0,0
10: LOD 1,12
11: LITI 0,1
12: LEQI 0,0
13: JPC 0,18
14: LDA 1,-4
15: LITI 0,1
16: STO 0,0
17: RET 0,0
18: LDA 1,-4
19: LOD 1,12
20: INT 0,16
21: LOD 1,12
22: LITI 0,1
23: SUBI 0,0
24: POP 0,4
25: ADDR 0,3
26: CAL 0,0
27: MULI 0,0
28: STO 0,0
29: RET 0,0
30: RET 0,0
31: INT 0,16
32: LDA 1,-4
33: LITI 0,0
34: STO 0,0
35: RET 0,0
36: RET 0,0
start execution
end execution

```

[설명]

factorial에서는 파라미터를 printf로 출력하기 위한 function_call을 수행하고 조건문을 확인한 후에 거짓이면 return value 부분에 결과 값을 저장하는데, 온전히 저장되기 전에 재귀함수에서 조건을 만나 돌아오기 전까지 n-1의 파라미터로 factorial을 호출한다. 만약 참이라면 return value에 1을 저장하도록 설정되어 있다.

32.

[입력]

```
virtual-machine:~$ cat ex.c
int gcd(int x, int y){
    int a,b;
    a=x;
    b=y;
    while(a!=b){
        if(a<b) b=b-a;
        if(a>b) a=a-b;
    }
    return(a);
}

void main(){
    int i;
    i=gcd(84,36);
    printf("result=%d\n",i);
}
```

[출력값: 시멘틱 트리, 어셈블리 코드, 인터프리터 결과]

```
virtual-machine:~$ cat a.asm
      INT 0, 28
      SUP 0, main
      RET 0, 0
gcd:
      INT 0, 28
      LDA 1, 20
      LOD 1, 12
      STX 0, 0
      POP 0, 1
      LDA 1, 24
      LOD 1, 16
      STX 0, 0
      POP 0, 1
L2:
      LOD 1, 20
      LOD 1, 24
      NEQI 0, 0
      JPC 0, L3
      LOD 1, 20
      LOD 1, 24
      LSSI 0, 0
      JPC 0, L4
      LDA 1, 24
      LOD 1, 24
      LOD 1, 20
      SUBI 0, 0
      STX 0, 0
      POP 0, 1
L4:
      LOD 1, 20
      LOD 1, 24
      GTRI 0, 0
      JPC 0, L5
      LDA 1, 20
      LOD 1, 20
      LOD 1, 24
      SUBI 0, 0
      STX 0, 0
      POP 0, 1
L5:
L1:
      JMP 0, L2
L3:
      LDA 1, -4
      LOD 1, 20
      STO 0, 0
      RET 0, 0
      RET 0, 0
main:
      INT 0, 16
      LDA 1, 12
      INT 0, 16
      LITI 0, 84
      LITI 0, 36
      POP 0, 5
      ADDR 0, gcd
      CAL 0, 0
      STX 0, 0
      POP 0, 1
      INT 0, 12
      LDA 0, 12
      LOD 1, 12
      POP 0, 5
      ADDR 0, printf
      CAL 0, 0
      RET 0, 0
.literal 12 "result=%d\n"
```

```
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 42
5: gcd 3
6: L2 12
7: L3 37
8: L4 26
9: L5 36
10: L1 36
===== code =====
0: INT 0,28
1: SUP 0,42
2: RET 0,0
3: INT 0,28
4: LDA 1,20
5: LOD 1,12
6: STX 0,0
7: POP 0,1
8: LDA 1,24
9: LOD 1,16
10: STX 0,0
11: POP 0,1
12: LOD 1,20
13: LOD 1,24
14: NEQI 0,0
15: JPC 0,37
16: LOD 1,20
17: LOD 1,24
18: LSSI 0,0
19: JPC 0,26
20: LDA 1,24
21: LOD 1,24
22: LOD 1,20
23: SUBI 0,0
24: STX 0,0
25: POP 0,1
26: LOD 1,20
27: LOD 1,24
28: GTRI 0,0
29: JPC 0,36
30: LDA 1,20
31: LOD 1,20
32: LOD 1,24
33: SUBI 0,0
34: STX 0,0
35: POP 0,1
36: JMP 0,12
37: LDA 1,-4
38: LOD 1,20
39: STO 0,0
40: RET 0,0
41: RET 0,0
42: INT 0,16
43: LDA 1,12
44: INT 0,16
45: LITI 0,84
46: LITI 0,36
47: POP 0,5
48: ADDR 0,3
49: CAL 0,0
50: STX 0,0
51: POP 0,1
52: INT 0,12
53: LDA 0,12
54: LOD 1,12
55: POP 0,5
56: ADDR 0,-1
57: CAL 0,0
58: RET 0,0
start execution
result=12
end execution
```

[설명]

교재 p.203에 나와 있는 최소 공배수를 구하는 코드로 구조체, 공용체, switch를 사용하지 않아 stx를 0,0으로 설정한 것과 label 외에 거의 동일하고, 같은 결과를 출력한다. 시멘틱 트리도 적절하게 배치되어 있고 속성들도 잘 설정되어 있다. 내부의 어셈블리 코드를 확인해 보아 을 때, 주소 설정과 값의 치환 연산, while문, if문, return문, function_call이 적절하게 실행되고 있으며 점프해야하는 위치도 적절하게 설정되어 있다. 또한, 그 외의 수식들도 같은 int 타입의 변수를 연산하는 것을 확인할 수 있다.

33.

[입력]

```
-virtual-machine:~$ cat ex.c
main(){
    int n,x,i,k,lim,prim;
    int p[300];
    p[1]=2; printf("%d", 2);
    n=100;
    x=1;
    lim=1;
    for(i=2;i<=n;i++){
        do{
            x=x+1; x++;
            if(p[lim]*p[lim]<=x) lim++;
            k=2;
            prim=1;
            while(prim && (k<lim)){
                prim=x%p[k];
                k++;
            }
        } while(!prim);
        p[i]=x;
        printf("%d", x);
        if(i%10==0) printf("\n");
    }
}
```


[출력값: 어셈블리 코드, 인터프리터 결과]

```
-virtual-machine:~$ cat a.asm
    INT 0, 36
    SUP 0, main
    RET 0, 0
main:
    INT 0, 1236
    LDA 1, 36
    LITI 0, 1
    LITI 0, 4
    MULI 0, 0
    OFFSET 0, 0
    LITI 0, 2
    STX 0, 0
    POP 0, 1
    INT 0, 12
    LDA 0, 12
    LITI 0, 2
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0
    LDA 1, 12
    LITI 0, 100
    STX 0, 0
    POP 0, 1
    LDA 1, 16
    LITI 0, 1
    STX 0, 0
    POP 0, 1
    LDA 1, 28
    LITI 0, 1
    STX 0, 0
    POP 0, 1
    LDA 1, 20
    LITI 0, 2
    STX 0, 0
    POP 0, 1
L2:
    LOD 1, 20
    LOD 1, 12
    LEQI 0, 0
    JPC 0, L3
L6:
    LDA 1, 16
```

```

L6:
    LDA 1, 16
    LOD 1, 16
    LITI 0, 1
    ADDI 0, 0
    STX 0, 0
    POP 0, 1
    LOD 1, 16
    LDA 1, 16
    LDX 0, 1
    INCI 0, 0
    STO 0, 0
    POP 0, 1
    LDA 1, 36
    LOD 1, 28
    LITI 0, 4
    MULI 0, 0
    OFFSET 0, 0
    LDI 0, 0
    LDA 1, 36
    LOD 1, 28
    LITI 0, 4
    MULI 0, 0
    OFFSET 0, 0
    LDI 0, 0
    MULI 0, 0
    LOD 1, 16
    LEQI 0, 0
    JPC 0, L7
    LOD 1, 28
    LDA 1, 28
    LDX 0, 1
    INCI 0, 0
    STO 0, 0
    POP 0, 1

L7:
    LDA 1, 24
    LITI 0, 2
    STX 0, 0
    POP 0, 1
    LDA 1, 32
    LITI 0, 1
    STX 0, 0
    POP 0, 1

L9:
    LOD 1, 32
    JPCR 0, L10
    LOD 1, 24
    LOD 1, 28
    LSSI 0, 0

L10:
    JPC 0, L11
    LDA 1, 32
    LOD 1, 16
    LDA 1, 36
    LOD 1, 24
    LITI 0, 4
    MULI 0, 0
    OFFSET 0, 0
    LDI 0, 0
    MOD 0, 0
    STX 0, 0
    POP 0, 1
    LOD 1, 24
    LDA 1, 24
    LDX 0, 1
    INCI 0, 0
    STO 0, 0
    POP 0, 1

L8:

```

```

                POP  0, 1
L8:
                JMP  0, L9
L11:
L5:
                LOD  1, 32
                NOT  0, 0
                JPT  0, L6
L4:
                LDA  1, 36
                LOD  1, 20
                LITI 0, 4
                MULI 0, 0
                OFFSET 0, 0
                LOD  1, 16
                STX  0, 0
                POP  0, 1
                INT  0, 12
                LDA  0, 20
                LOD  1, 16
                POP  0, 5
                ADDR 0, printf
                CAL  0, 0
                LOD  1, 20
                LITI 0, 10
                MOD  0, 0
                LITI 0, 0
                EQLI 0, 0
                JPC  0, L12
                INT  0, 12
                LDA  0, 28
                POP  0, 4
                ADDR 0, printf
                CAL  0, 0
L12:
L1:
                LOD  1, 20
                LDA  1, 20
                LDX  0, 1
                INCI 0, 0
                STO  0, 0
                POP  0, 1
                JMP  0, L2
L3:
                RET  0, 0
.literal      12  "%d "
.literal      20  "%d "
.literal      28  "\n"

```

```
virtual-machine:~$ ./interpreter a.asm
===== symbol =====
1: printf -1
2: malloc -2
3: scanf -3
4: main 3
5: L2 34
6: L3 139
7: L6 38
8: L7 72
9: L9 80
10: L10 85
11: L11 104
12: L8 103
13: L5 104
14: L4 107
15: L12 132
16: L1 132
===== code =====
0: INT 0,36
1: SUP 0,3
2: RET 0,0
3: INT 0,1236
4: LDA 1,36
5: LITI 0,1
6: LITI 0,4
7: MULI 0,0
8: OFFSET 0,0
9: LITI 0,2
10: STX 0,0
11: POP 0,1
12: INT 0,12
13: LDA 0,12
14: LITI 0,2
15: POP 0,5
16: ADDR 0,-1
17: CAL 0,0
18: LDA 1,12
19: LITI 0,100
20: STX 0,0
21: POP 0,1
22: LDA 1,16
23: LITI 0,1
24: STX 0,0
25: POP 0,1
26: LDA 1,28
27: LITI 0,1
28: STX 0,0
29: POP 0,1
30: LDA 1,20
31: LITI 0,2
32: STX 0,0
33: POP 0,1
34: LOD 1,20
35: LOD 1,12
36: LEQI 0,0
37: JPC 0,139
38: LDA 1,16
39: LOD 1,16
40: LITI 0,1
41: ADDI 0,0
42: STX 0,0
43: POP 0,1
44: LOD 1,16
45: LDA 1,16
```

45:	LDA	1,16
46:	LDX	0,1
47:	INCI	0,0
48:	STO	0,0
49:	POP	0,1
50:	LDA	1,36
51:	LOD	1,28
52:	LITI	0,4
53:	MULI	0,0
54:	OFFSET	0,0
55:	LDI	0,0
56:	LDA	1,36
57:	LOD	1,28
58:	LITI	0,4
59:	MULI	0,0
60:	OFFSET	0,0
61:	LDI	0,0
62:	MULI	0,0
63:	LOD	1,16
64:	LEQI	0,0
65:	JPC	0,72
66:	LOD	1,28
67:	LDA	1,28
68:	LDX	0,1
69:	INCI	0,0
70:	STO	0,0
71:	POP	0,1
72:	LDA	1,24
73:	LITI	0,2
74:	STX	0,0
75:	POP	0,1
76:	LDA	1,32
77:	LITI	0,1
78:	STX	0,0
79:	POP	0,1
80:	LOD	1,32
81:	JPCR	0,85
82:	LOD	1,24
83:	LOD	1,28
84:	LSSI	0,0
85:	JPC	0,104
86:	LDA	1,32
87:	LOD	1,16
88:	LDA	1,36
89:	LOD	1,24
90:	LITI	0,4
91:	MULI	0,0
92:	OFFSET	0,0
93:	LDI	0,0
94:	MOD	0,0
95:	STX	0,0
96:	POP	0,1
97:	LOD	1,24
98:	LDA	1,24
99:	LDX	0,1
100:	INCI	0,0
101:	STO	0,0
102:	POP	0,1
103:	JMP	0,80
104:	LOD	1,32
105:	NOT	0,0
106:	JPT	0,38
107:	LDA	1,36
108:	LOD	1,20
109:	LITI	0,4
110:	MULI	0,0
111:	OFFSET	0,0
112:	LOD	1,16

```

112: LOD      1,16
113: STX      0,0
114: POP      0,1
115: INT      0,12
116: LDA      0,20
117: LOD      1,16
118: POP      0,5
119: ADDR     0,-1
120: CAL      0,0
121: LOD      1,20
122: LITI     0,10
123: MOD      0,0
124: LITI     0,0
125: EQLI     0,0
126: JPC      0,132
127: INT      0,12
128: LDA      0,28
129: POP      0,4
130: ADDR     0,-1
131: CAL      0,0
132: LOD      1,20
133: LDA      1,20
134: LDX      0,1
135: INCI     0,0
136: STO      0,0
137: POP      0,1
138: JMP      0,34
139: RET      0,0
start execution
2 3 5 7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97 101 103 107 109 113
127 131 137 139 149 151 157 163 167 173
179 181 191 193 197 199 211 223 227 229
233 239 241 251 257 263 269 271 277 281
283 293 307 311 313 317 331 337 347 349
353 359 367 373 379 383 389 397 401 409
419 421 431 433 439 443 449 457 461 463
467 479 487 491 499 503 509 521 523 541
end execution

```

[설명]

교재 p.204에 있는 프로그램으로, 배열을 처리할 때 사용되는 offset이 적절하게 설정되어 있고, int 타입 변수들의 연산에 맞게 연산의 타입이 int인 것이 확인된다. 그리고 함수의 파라미터 설정과 function_call이 잘 동작한다. 또한 for문과 do-while문, while문, if문 역시도 적절하게 작동한다. 마지막으로 label의 설정과 jump의 위치도 적절한 것을 확인하였으며 교재의 출력 결과와 동일한 결과를 출력하는 것을 확인했다.

34.

[입력]

```
virtual-machine:~$ cat ex.c
int cnt[123];

int main()
{
    int a,i,j;
    a=10;
    i=0;

    while(1){
        cnt[i]=a%2;
        a=a/2;
        if(a==0)break;
        i++;
    }

    for(j=i;j>=0;j--){
        printf("%d",cnt[j]);
    }
    printf("\n");
    return 0;
}
```

[출력값: 어셈블리 코드, 인터프리터 결과]

```
virtual-machine:~$ cat a.asm
    INT 0, 520
    SUP 0, main
    RET 0, 0
main:
    INT 0, 24
    LDA 1, 12
    LITI 0, 10
    STX 0, 0
    POP 0, 1
    LDA 1, 16
    LITI 0, 0
    STX 0, 0
    POP 0, 1
L2:
    LITI 0, 1
    JPC 0, L3
    LDA 0, 12
    LOD 1, 16
    LITI 0, 4
    MULI 0, 0
    OFFSET 0, 0
    LOD 1, 12
    LITI 0, 2
    MOD 0, 0
    STX 0, 0
    POP 0, 1
    LDA 1, 12
    LOD 1, 12
    LITI 0, 2
    DIVI 0, 0
    STX 0, 0
    POP 0, 1
    LOD 1, 12
    LITI 0, 0
    EQLI 0, 0
    JPC 0, L4
    JMP 0, L3
L4:
```

```

L4:
    LOD 1, 16
    LDA 1, 16
    LDX 0, 1
    INCI 0, 0
    STO 0, 0
    POP 0, 1

L1:
    JMP 0, L2

L3:
    LDA 1, 20
    LOD 1, 16
    STX 0, 0
    POP 0, 1

L6:
    LOD 1, 20
    LITI 0, 0
    GEQI 0, 0
    JPC 0, L7
    INT 0, 12
    LDA 0, 504
    LDA 0, 12
    LOD 1, 20
    LITI 0, 4
    MULI 0, 0
    OFFSET 0, 0
    LDI 0, 0
    POP 0, 5
    ADDR 0, printf
    CAL 0, 0

L5:
    LOD 1, 20
    LDA 1, 20
    LDX 0, 1
    DECI 0, 0
    STO 0, 0
    POP 0, 1
    JMP 0, L6

L7:
    INT 0, 12
    LDA 0, 512
    POP 0, 4
    ADDR 0, printf
    CAL 0, 0
    LDA 1, -4
    LITI 0, 0
    STO 0, 0
    RET 0, 0
    RET 0, 0

.literal 504 "%d"
.literal 512 "\n"

```

```
-virtual-machine:~$ ./interpreter a.asm
===== symbol =====
 1: printf  -1
 2: malloc  -2
 3: scanf   -3
 4: main     3
 5: L2       12
 6: L3       42
 7: L4       35
 8: L1       41
 9: L6       46
10: L7       68
11: L5       61
===== code =====
 0: INT      0,520
 1: SUP      0,3
 2: RET      0,0
 3: INT      0,24
 4: LDA      1,12
 5: LITI     0,10
 6: STX      0,0
 7: POP      0,1
 8: LDA      1,16
 9: LITI     0,0
10: STX      0,0
11: POP      0,1
12: LITI     0,1
13: JPC      0,42
14: LDA      0,12
15: LOD      1,16
16: LITI     0,4
17: MULI     0,0
18: OFFSET   0,0
19: LOD      1,12
20: LITI     0,2
21: MOD      0,0
22: STX      0,0
23: POP      0,1
24: LDA      1,12
25: LOD      1,12
26: LITI     0,2
27: DIVI     0,0
28: STX      0,0
29: POP      0,1
30: LOD      1,12
31: LITI     0,0
32: EQLI     0,0
33: JPC      0,35
```

```

33: JPC      0,35
34: JMP      0,42
35: LOD      1,16
36: LDA      1,16
37: LDX      0,1
38: INCI     0,0
39: STO      0,0
40: POP      0,1
41: JMP      0,12
42: LDA      1,20
43: LOD      1,16
44: STX      0,0
45: POP      0,1
46: LOD      1,20
47: LITI     0,0
48: GEQI     0,0
49: JPC      0,68
50: INT      0,12
51: LDA      0,504
52: LDA      0,12
53: LOD      1,20
54: LITI     0,4
55: MULI     0,0
56: OFFSET   0,0
57: LDI      0,0
58: POP      0,5
59: ADDR     0,-1
60: CAL      0,0
61: LOD      1,20
62: LDA      1,20
63: LDX      0,1
64: DECI     0,0
65: STO      0,0
66: POP      0,1
67: JMP      0,46
68: INT      0,12
69: LDA      0,512
70: POP      0,4
71: ADDR     0,-1
72: CAL      0,0
73: LDA      1,-4
74: LITI     0,0
75: STO      0,0
76: RET      0,0
77: RET      0,0
start execution
1010
end execution

```

[설명]

전역변수 설정과 지역 변수 설정이 제대로 되어 있는 것을 확인할 수 있고, 배열의 offset 계산과 MOD, DIV, EQL, QEQ 연산이 잘 수행되었다. 또한, while문과 for문에서 jump 해야 할 label의 설정과 흐름이 적절하다. 최종적으로 출력되는 결과 역시도 적절한 결과가 출력된다.